

What a Reader Recovers

Access Regimes, Contingency Tables, and the Handedness Threshold for Finite Transformations

Driven by Dean A. Kulik

September 2026

Abstract

An observer interrogating an unknown transformation does not recover the transformation. It recovers whichever equivalence class survives its access constraints, and the class depends on what the observer is permitted to do rather than on how much of it it does. This paper fixes a finite model in which that dependence is computed exactly and, in the central cases, proved: a permutation σ of a finite set, an observer whose resolution is a partition P , and four access regimes running from passive trajectory observation through single-step intervention to trajectory-tracking intervention.

The organising identification is elementary once stated. The record of a single-step intervening reader is exactly the contingency table $N(P, \sigma)$ with entries $N_{ij} = |B_i \cap \sigma^{-1}B_j|$, so the recovered object is the corresponding double coset $K\sigma K$, where K is the Young subgroup of within-block relabellings, and its size is the classical contingency count $(\prod |B_i|!)^2 / \prod N_{ij}!$. This puts the whole problem inside the well-studied correspondence between parabolic double cosets in the symmetric group and two-way tables with prescribed margins, and it converts every question below into a question about tables.

Four results follow. First, an inversion criterion: σ^{-1} is indistinguishable from σ at resolution P if and only if $N(P, \sigma)$ is symmetric — verified with zero counterexamples across 152,760 configurations. Since a two-block table has equal row and column margins and is therefore always symmetric, no family of binary readers can ever distinguish a transformation from its inverse under single-step or iterated access, however many binary readers are deployed.

Second, the complete binary family recovers exactly the undirected functional graph of σ and nothing more, so its candidate set is the set of orientations of that graph, of size 2^c where c is the number of cycles of length at least three. This is proved and confirmed on every cycle type for $n = 4$ through 7.

Third, a handedness threshold. Trajectory access can break the orientation tie, but only when the reader's labels form a chiral cyclic word: a binary reader distinguishes a k -cycle from its inverse if and only if some binary necklace of length k differs from its reversal, which happens if and only if $k \geq 6$. Below that length the handedness of a cycle is undetectable at binary resolution under every protocol in the hierarchy. The computed thresholds match the classical necklace-versus-bracelet counts exactly.

Fourth, exact recovery is a property of reader families rather than of resolution. A single reader recovers σ exactly only when fully labelled, but two proper partitions suffice for every cycle type tested at $n = 5$ and $n = 6$, and one never does. Coarse readers with differently shaped labels break one another's gauge.

Underneath the hierarchy sits an identity. The trajectory reader candidate set has exactly $|K|$ elements for every transformation, because the multinomial distributing itinerary words among states cancels against the freedom inside each equal-word class, and the transformation drops out. Two consequences follow at once: the gauge divides the candidate set by orbit-stabiliser, and a reader extracts everything its labels permit exactly when the transformation commutes with no nontrivial within-block relabelling, which accounts for the observed 54% rate of gauge attainment without residue.

Along the way an earlier protocol-dependent result is corrected: single-step access carries most of the ambiguity usually attributed to coarse resolution, strictly so in 82% of configurations at $n = 6$, and a null model that appeared to show two readers combining super-additively is retired, its excess being unbounded along an explicit family and therefore a property of double-coset geometry rather than of measurement. The double-coset and contingency-table machinery is classical and is cited, not claimed.

Contents

Abstract	1
Contents	2
1. The question.....	4
2. The model.....	4
2.1 Four access regimes.....	5
2.2 The gauge.....	5
2.3 Families	5
3. The contingency dictionary	5
4. The inversion criterion.....	6
4A. A worked example	7
5. The ladder	7
6. Floors	11
6.1 The single-step floor is set by the labels alone.....	11
6.2 The floor for coarse access as such is lower	11
6.3 The $\mathbb{Z}/12$ instrument, read completely	12
6A. The sampling horizon	12
6A.1 What this says about sampling	13
7. Passive observation and the exchange rate	14

8. Two readers, and a null model that fails	14
8.1 The excess is unbounded and the baseline is wrong	15
9. Families of readers: what binary access recovers.....	16
10. The handedness threshold.....	17
10.1 Iteration does not help	17
10.2 Trajectory access, and when it helps	17
10.3 What this settles in the wider programme.....	18
11. Exact recovery is a property of families	18
11.2 How common are sufficient pairs	19
11A. Binary is the only failing resolution	19
11B. A reader that runs.....	20
12. What this says about inferring symmetry	22
13. Status	22
14. What this does not establish.....	24
14A. Related work.....	25
15. Reproduction.....	25
16. Open problems.....	26

1. The question

A transformation acts on a finite set of states. An observer wishes to learn it but cannot see states — only labels, with several states sharing a label. What can the observer conclude?

The tempting formulation of the answer is quantitative: the observer learns some number of bits about a fixed object, and more access yields more bits. That formulation is wrong in a way this paper makes precise. Different access regimes do not deliver different amounts of information about one object; they deliver different objects. A passive observer and an intervening observer with identical label resolution hold sets that are not nested, not consistently ordered by size, and not related by any inclusion of information content. Passive observation of a small-orbit transformation beats coarse intervention outright; coarse intervention on a transitive one beats passive observation from a single bit of labelling.

Two hazards sit in the way of answering the question honestly, and both are addressed before any result is stated.

The first is the protocol. The natural first formalisation restricts the intervening reader to a single application, with each preparation drawn afresh from within its label block — a reasonable model of an apparatus that cannot hold a state between runs. It yields a clean algebraic answer, and it is also responsible for most of the ambiguity it reports. Admitting repeated applications strictly shrinks the candidate set in 1,838 of 2,233 configurations at $n = 6$. Any conclusion drawn from the single-step model alone is a statement about the apparatus and must be labelled as such. Section 6 measures the gap before the later sections rely on what survives it.

The second is the count of readers. One reader has a gauge — a group of relabellings it cannot see through — and the natural conclusion is that exact recovery requires full labelling. That is true of one reader and false of a family, and the difference turns out to carry the paper's sharpest results. Sections 10 through 12 are about families.

The mathematics is standard throughout. Double cosets of Young (parabolic) subgroups of the symmetric group are classical, with a well-developed enumeration literature and a bijection to two-way contingency tables with prescribed margins going back to Diaconis and Gangolli. Coarse-grained observation of a dynamics is lumping, and identifiability under lumping has a long history. Binary necklaces and bracelets are classical enumerative objects. None of that is claimed. What is claimed is the identification of the reader's recovered object with these structures, the exact prices the identification makes available, and the handedness threshold that falls out of it.

2. The model

Let X be a finite set with $|X| = n$, and let $\sigma \in \text{Sym}(X)$ be the unknown transformation. A reader is specified by a partition $P = \{B_1, \dots, B_m\}$ of X , its label resolution, and we write

$$K = K(P) = \prod_i S_{|B_i|} \leq \text{Sym}(X)$$

for the Young subgroup of permutations preserving every block setwise — the relabellings the reader cannot detect. Every quantity in this paper is a function of K , σ and the access regime.

2.1 Four access regimes

R_0 , passive observation. The reader applies nothing and watches the system evolve, recovering the orbit structure of $\langle \sigma \rangle$ — for a permutation, the cycle partition.

R_1 , single-step intervention. The reader prepares a state specified only by its label and reads the label of its image. The label does not determine the state, so each preparation is a fresh draw from within its block, and the reader's complete record is, for each block, the multiset of destination labels.

R_2 , iterated intervention. As R_1 , but the reader may apply σ repeatedly before reading, obtaining the analogous record for $\sigma, \sigma^2, \sigma^3$, and so on. Preparations remain fresh draws, so identity is not carried between runs.

R_3 , trajectory intervention. The reader prepares a state and reads the label at every subsequent step of that single run, obtaining for each starting state a label itinerary. Identity is carried within a run but not between runs.

Write $R_1(\sigma, P) \supseteq R_2(\sigma, P) \supseteq R_3(\sigma, P)$ for the sets of transformations consistent with each record. The inclusions are immediate, since each regime's record refines the previous one's.

2.2 The gauge

Beneath all of them lies a set no coarse regime penetrates. Relabelling states within a block is invisible to every reader specified by P , so for any $\kappa \in K$ the transformation $\kappa\sigma\kappa^{-1}$ produces an identical record in every regime. Hence

$$G(\sigma, P) = \{ \kappa\sigma\kappa^{-1} : \kappa \in K \}, \quad |G| = |K| / |C_K(\sigma)|,$$

and $G \subseteq R_3 \subseteq R_2 \subseteq R_1$. G is the reader's gauge: irreducible ambiguity arising from the labels themselves rather than from any deficiency of protocol. A reader attaining G has extracted everything its resolution permits.

2.3 Families

A reader family is a finite set of partitions $P_1, \dots, P_r$, and its candidate set is the intersection $\bigcap_i R_i(\sigma, P_i)$ in whichever regime i is under discussion. Nothing forces the family's gauges to coincide, and Sections 10 to 12 are about what happens when they do not.

3. The contingency dictionary

The R_1 record was defined operationally, and its consistent set is not obviously algebraic. It is, and the identification is the organising fact of the paper.

Definition 3.1.

For a partition P and a permutation σ , the contingency table of σ at resolution P is the $m \times m$ matrix $N(P, \sigma)$ with entries $N_{ij} = |\{x \in B_i : \sigma x \in B_j\}| = |B_i \cap \sigma^{-1}B_j|$. Its row sums are $|B_i|$ and, because σ is a bijection, its column sums are $|B_j|$. Row and column margins therefore coincide.

Proposition 3.2.

The R_1 record of σ at resolution P is exactly $N(P, \sigma)$. The record specifies, for each block, the multiset of destination labels, which is precisely row i of the table; conversely the table lists nothing else.

Theorem 3.3.

The set of transformations consistent with the R_1 record of σ at resolution P is the double coset $K\sigma K$. Equivalently, two permutations lie in the same K -double coset if and only if they have the same contingency table at resolution P .

The second formulation is the classical parametrisation of parabolic double cosets by two-way tables with prescribed margins. The first was verified operationally rather than assumed, by exhaustive enumeration of $\text{Sym}(X)$ at $n = 6$ against the record as defined:

$P = \{012|345\}$, $\sigma = +1$: $|\text{consistent}| = 324$ $|K\sigma K| = 324$ closed form = 324
 $P = \{03|14|25\}$, $\sigma = +1$: $|\text{consistent}| = 8$ $|K\sigma K| = 8$ closed form = 8
 $P = \{01|23|45\}$, $\sigma = +2$: $|\text{consistent}| = 8$ $|K\sigma K| = 8$ closed form = 8
 $P = \{012|345\}$, $\sigma = +3$: $|\text{consistent}| = 36$ $|K\sigma K| = 36$ closed form = 36
 $P = \{024|135\}$, $\sigma = +1$: $|\text{consistent}| = 36$ $|K\sigma K| = 36$ closed form = 36

Corollary 3.4 (the price of a resolution).

The ambiguity of a single-step reader is

$$|R_1(\sigma, P)| = |K\sigma K| = (\prod_i |B_i|!)^2 / \prod_{i,j} N_{ij}!,$$

computable from block sizes and one table, with no enumeration. Verified against direct construction of the double coset in all 6,600 partition-permutation pairs at $n = 4$ and $n = 5$.

The formula also explains a fact used repeatedly below. The standard identity $|K\sigma K| = |K|^2 / |K \cap \sigma K \sigma^{-1}|$ combines with the observation that $\sigma K \sigma^{-1}$ is the Young subgroup of the image partition σP , and that the intersection of two Young subgroups is the Young subgroup of the meet of their partitions. The blocks of that meet are exactly the sets $B_i \cap \sigma^{-1}B_j$, whose sizes are the table entries — so the meet formula and the contingency formula are the same statement written twice.

4. The inversion criterion

With the dictionary in place, the question of when a reader can tell a transformation from its inverse becomes a question about matrix transposes, and it answers itself.

Lemma 4.1.

$N(P, \sigma^{-1}) = N(P, \sigma)^T$. Immediately from the definition: $|B_i \cap \sigma B_j| = |B_j \cap \sigma^{-1}B_i|$.

Theorem 4.2 (inversion criterion).

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828
 Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

$\sigma^{-1} \in K\sigma K$ if and only if $N(P, \sigma)$ is symmetric. By Theorem 3.3 the double coset is determined by the table, and by Lemma 4.1 the inverse's table is the transpose; the two coincide exactly when the table is symmetric. Verified with zero counterexamples across all 152,760 triples (n, P, σ) for $n = 4, 5, 6$.

Corollary 4.3 (binary readers are inversion-blind).

If $|P| \leq 2$ then $N(P, \sigma)$ is symmetric for every σ , so $\sigma^{-1} \in K\sigma K$ always. A 2×2 table has row margins (r_1, r_2) equal to its column margins, so with $N_{11} = a$ the remaining entries are $r_1 - a$, $r_1 - a$ and $r_2 - r_1 + a$: the off-diagonal entries are forced equal and the table is symmetric. All 25,152 triples with $|P| \leq 2$ confirm it.

The corollary is the seed of Sections 10 and 11. No single binary reader can distinguish a transformation from its inverse, and since the argument is about margins rather than about which subset was chosen, no collection of binary readers can either.

4A. A worked example

The dictionary is easiest to see on one transformation read two ways. Take $X = \{0, \dots, 5\}$ and σ the 6-cycle sending each x to $x+1 \bmod 6$. Two readers are given the same resolution — two blocks of three — and differ only in how the blocks are cut.

Reader A uses $P = \{0, 1, 2 \mid 3, 4, 5\}$. Four of the six steps stay inside a block and two cross, giving

$$N(P_A, \sigma) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad |K| = 3! \cdot 3! = 36, \quad |K\sigma K| = 36^2 / (2! \cdot 1! \cdot 1! \cdot 2!) = 324.$$

Reader B uses $P = \{0, 2, 4 \mid 1, 3, 5\}$. Every step crosses, giving

$$N(P_B, \sigma) = \begin{bmatrix} 0 & 3 \\ 3 & 0 \end{bmatrix}, \quad |K| = 36, \quad |K\sigma K| = 36^2 / (3! \cdot 3!) = 36.$$

Both counts were confirmed by direct enumeration in Section 3. The readers have identical resolution and their ambiguities differ by $\log_2(324/36) = 3.17$ bits. Reader B is aligned — σ carries each of its blocks onto the other — so by Section 6.1 it attains the floor $\log_2|K| = 5.17$ bits exactly, and Reader A sits 3.17 bits above it. Nothing about the number of labels distinguishes them; only the relation between the cut and the transformation does.

Both tables are symmetric, as Corollary 4.3 requires of any two-block reader, so neither reader can tell σ from σ^{-1} — and by Proposition 10.1 neither can tell them apart after any number of iterations. Reader B's label word around the cycle is 010101, which reads the same backwards; Reader A's is 001001, likewise. Both are achiral, so under trajectory access they still cannot separate the two orientations. At $n = 6$ a chiral word exists — 001011 — and the reader using it does separate them. That single word is the entire difference between an instrument that sees handedness and one that does not.

5. The ladder

The four regimes were measured exhaustively up to symmetry. Because conjugate transformations yield double cosets related by relabelling the partition, a census over one representative per cycle type against every set partition is complete rather than sampled.

n	configurations	$ R_2 < R_1 $	$ R_3 < R_2 $	$R_3 = G$	$R_1 = G$
4	75	40	3	39 (52.0%)	5
5	364	262	18	201 (55.2%)	7
6	2,233	1,838	132	1,205 (54.0%)	11

Three readings, in descending order of importance.

The single-step restriction carries most of the ambiguity. Admitting repeated application strictly shrinks the candidate set in 82.3% of configurations at $n = 6$. The natural first model is therefore not a conservative approximation of coarse access; it is a substantially weaker reader, and results derived from it alone describe an apparatus rather than a resolution.

Tracking identity within a run adds little once iteration is allowed. R_3 is strictly smaller than R_2 in only 5.9% of configurations at $n = 6$. Almost everything a trajectory reader knows is already available to a reader that merely repeats the experiment. The distinction between carrying identity through a run and running the experiment again is, in this model, nearly free — which makes the cases where it is not free (Section 11) the interesting ones.

The gauge is attained about half the time: $R_3 = G$ in 54.0% of configurations at $n = 6$, with the proportion stable across $n = 4, 5, 6$. In the remainder, the strongest coarse reader sits strictly above its own irreducible floor, meaning there exist transformations not K-conjugate to σ that produce an identical labelled trajectory record. That R_1 attains G in only 11 of 2,233 configurations shows how far the single-step reader sits from the floor.

Ambiguity therefore has three distinguishable sources, and the census separates them: protocol ambiguity (R_1 against R_2), resolution ambiguity (the gauge G), and structural indistinguishability (R_3 strictly above G).

The ladder has a bottom rung of fixed size, and this was found only after the divisibility of $|R_3|$ by $|G|$ had been recorded as an unexplained observation and then reduced to a case split. Both were unnecessary. The truth is simpler and is an identity.

Lemma 5.0 (realisation).

Fix an assignment of itinerary words to states that respects initial labels, so each state x receives a word $w(x)$ beginning with the label of x . For each word w let S_w be the set of states assigned w , and choose a bijection from S_w onto $S_{\{Sw\}}$, where S denotes the shift that deletes the first symbol. These choices determine a unique permutation τ , and for every state the assigned word is exactly its τ -itinerary.

The verification is mechanical. The chosen bijections define τ on all of X and τ is a bijection because the multiset is shift-invariant, so $|S_w| = |S_{\{Sw\}}|$ for every w . By construction the label of $\tau(x)$ is the first symbol of $S(w(x))$, which is the second symbol of $w(x)$; iterating, the k -th symbol of $w(x)$ is the label of $\tau^k(x)$, so the τ -itinerary of x is $w(x)$. Conversely, a permutation τ in R_3 determines its assignment, namely $w(x) =$ the τ -itinerary of x , and determines each bijection, namely the restriction of τ to S_w . The two constructions are mutually inverse, so R_3 is in bijection with the set of pairs (assignment, family of bijections). QED

Theorem 5.1 (the trajectory reader ambiguity equals its own relabelling group).

For every sigma and every partition P, $|R_3(\text{sigma}, P)| = |K(P)|$ = the product over blocks of $|B_i|!$. The size of the trajectory candidate set does not depend on the transformation at all.

Proof. Let W be the multiset of label itineraries of sigma, with m_w states carrying the word w. Since sigma permutes X, W is shift-invariant as a multiset and m_w equals m of the shifted word. By Lemma 5.0, R_3 is in bijection with pairs consisting of an assignment and a family of bijections. Each state must receive a word beginning with its own label, and the words beginning with label i have total multiplicity $|B_i|$, so the assignments number the product of $|B_i|!$ divided by the product of $m_w!$. The bijections are free and number the product of $m_w!$. The two factors cancel. QED

$$|R_3| = (\prod |B_i|! / \prod m_w!) \times \prod m_w! = \prod |B_i|! = |K|$$

The multinomial from distributing words among states cancels exactly against the freedom inside each equal-word class, which is why the transformation drops out. Verified term by term — assignment count, bijection count, and product — in all 2,672 configurations at $n = 4, 5, 6$, with the identity confirmed at itinerary horizons $n, 2n, 4n$ and $8n$ and at $n = 7$, so it is not an artifact of truncation.

Corollary 5.2.

$|G|$ divides $|R_3|$, immediately, since $|G| = |K|/|C_K(\text{sigma})|$ divides $|K| = |R_3|$ by orbit-stabiliser. The multiplicities observed at $n = 6$ — 1, 2, 3, 4, 5, 6, 8, 9, 12, 16, 18, 24, 36, 48, 120 and 720 — are exactly the possible orders of $C_K(\text{sigma})$. The earlier split into equal and unequal class sizes describes how R_3 decomposes but is irrelevant to the count.

Corollary 5.3 (when a reader attains its gauge).

$R_3 = G$ if and only if $C_K(\text{sigma})$ is trivial, since the sets are nested and their sizes are $|K|$ and $|K|/|C_K(\text{sigma})|$. The strongest coarse reader therefore extracts everything its labels permit exactly when the transformation commutes with no nontrivial within-block relabelling. This accounts for the census column of Section 5 without residue: $C_K(\text{sigma})$ is trivial in 52.0%, 55.2% and 54.0% of configurations at $n = 4, 5, 6$, matching the observed rates of gauge attainment exactly.

Proposition 5.4 (when the ladder collapses).

The following are equivalent, verified as sets in all 2,672 configurations: P is aligned with sigma; $|R_1| = |K|$; $R_2 = R_1$; $R_3 = R_1$. Iteration and trajectory tracking buy nothing precisely when the labels are aligned, and the hierarchy collapses to a single rung there. Combined with Theorem 5.1 this says R_1 is at least $|K| = |R_3|$ always, with equality exactly under alignment — so the quantity $\log_2 |K|$ of Section 6 is not a floor the single-step reader sometimes reaches, but the trajectory reader value, universally.

Theorem 5.5 (structure of the trajectory residual).

Theorem 5.1 fixes the size of R_3 but not its shape, and the shape follows from the same parametrisation. Let the word-classes of sigma be grouped by the induced shift map, which sends the class of x to the class of $\text{sigma}(x)$; the classes in one orbit all have the same size, and let $m_1, \dots, m_r$ be those sizes, one per orbit. The K-conjugacy classes contained in R_3 are in bijection with tuples of conjugacy classes of $S_{m_1} \times \dots \times S_{m_r}$, and for tau corresponding to return maps $\rho_{01}, \dots, \rho_{or}$,

$$|C_K(\tau)| = \prod_j |C_{S_m}(\rho_j)|, \text{ class size} = |K| / \prod_j |C_{S_m}(\rho_j)|, \text{ number of classes} = \prod_j p(m_j),$$

with p the partition function.

Proof. Send each τ in R_3 to its own assignment A_τ , the function taking a state to its τ -itinerary. This map is K -equivariant. The group K acts transitively on assignments, because within a block S_{B_i} acts transitively on the ways of distributing that block's multiset of words among its states, and the stabiliser of an assignment A is the product over word-classes of the symmetric groups on those classes. So R_3 fibres K -equivariantly over a single K -orbit, and the K -orbits on R_3 correspond to the orbits of $\text{Stab}(A)$ on the fibre over one fixed A . Moreover $C_K(\tau)$ is contained in $\text{Stab}(A_\tau)$: if κ in K commutes with τ then, since κ preserves labels, the label of $\tau^k(\kappa x)$ equals the label of $\tau^k(x)$ for all k , so κ preserves τ -itineraries.

It remains to describe the $\text{Stab}(A)$ -orbits on the fibre. By Lemma 5.0 the fibre is the set of families of bijections, one along each edge of each shift-orbit of word-classes. Fix one orbit, a directed cycle of r classes each of size m . Its data is r bijections composing around the cycle, and $\text{Stab}(A)$ acts on them by relabelling each class independently. Relabelling can be used to set all but one of the bijections to a fixed reference, and the remaining freedom carries the composite once around the cycle. That composite, the return map ρ , is therefore the only surviving invariant, and the residual relabelling acts on it by conjugation in S_m . Orbits are conjugacy classes and stabilisers are centralisers, so orbits contribute $|C_{S_m}(\rho)|$ to the stabiliser order and the counts follow. Orbits are independent across shift-orbits, giving the product. QED

Both steps were checked before being written: K acts transitively on assignments with the stated stabiliser, and $C_K(\tau)$ lies in $\text{Stab}(A_\tau)$, in all 2,672 configurations at $n = 4, 5, 6$. The predicted multiset of class sizes and the predicted class count agree with the observed decomposition in the same 2,672 configurations. An earlier version of the containment step, asserting that centraliser elements preserve the word-classes of σ rather than of τ , is false and fails in 478 of those configurations; the distinction matters because τ need not assign the same word to a state that σ does.

The argument is a closure argument, and naming it as one is what produced it. The interior bijections along a shift-orbit are gauge: each can be altered by relabelling a class, and the alteration propagates to its neighbour without leaving a trace. Only on returning to the starting class is there something that can be compared with itself. The residual invariant lives at the closure of the loop, not on any of its edges, which is why the individual multiplicities m_w cancel and the return map survives.

Theorem 5.1 is a corollary. Summing class sizes gives $|K|$ times the product over orbits of the sum over conjugacy classes of S_m of the reciprocal centraliser order, and each such sum is 1 by the class equation. The identity $|R_3| = |K|$ is therefore a product of class equations of symmetric groups, one per shift-orbit of word-classes.

Corollary 5.6.

All K -classes in R_3 have the same size if and only if every word-class has size at most two. S_1 has one conjugacy class and S_2 has two whose centralisers both have order two, so the centraliser order is constant in those cases and varies from S_3 onward. This accounts for the earlier and now-superseded split into equal and unequal class sizes: it was a statement about whether any word-class had three or more members, and the profiles that looked irregular — (1, 15, 15, 40, 40, 45, 90, 90, 120, 120, 144) at $n = 6$ — are the conjugacy

class sizes of S_6 , arising when a single word-class contains all six states. Verified with no exceptions across the same 2,672 configurations.

6. Floors

6.1 The single-step floor is set by the labels alone

Since $|K \cap \sigma K \sigma^{-1}| \leq |K|$, we have $|K \sigma K| \geq |K|$ with equality exactly when σ normalises K — that is, when σ carries blocks onto blocks. Call such a partition aligned with σ . In bits,

$$\log_2 |K \sigma K| \geq \log_2 |K| = \sum_i \log_2 (|B_i|!),$$

with equality iff P is aligned. In table language the same statement reads: the table of an aligned reader is a permutation matrix scaled by block sizes, which maximises $\prod |N_{ij}|$ and therefore minimises the count of Corollary 3.4. The bound carries no information about σ and is identical at a given resolution for every transformation.

Labels	Floor $\log_2 K $	Median	Worst	Best-to-worst spread
2	18.984	26.797	27.628	$2^{8.644}$
3	13.755	23.510	24.510	$2^{10.755}$
4	10.340	18.680	20.680	$2^{10.340}$
6	6.000	12.000	12.000	$2^{6.000}$

Exhaustive sweeps over all uniform partitions of $\mathbb{Z}/12$ at four resolutions, for $\sigma \in \{+1, +2, +3\}$, attain the floor at every resolution, attain it precisely on the aligned partitions, and tie across all of them. Two readers with the same number of labels can sit more than ten bits apart purely because one has cut its labels across the transformation and the other along it. Resolution and alignment are independent quantities, and the second is not under the reader's control once the labels are fixed.

6.2 The floor for coarse access as such is lower

The bound of 6.1 applies to R_1 and must not be reported as the reader floor. By Theorem 5.1 the quantity $\log_2 |K|$ is exactly the trajectory reader ambiguity, for every transformation, so what 6.1 measures is the excess of the single-step reader over the trajectory reader: a protocol cost, not a resolution cost. The floor for coarse access as such is lower, namely $|G| = |K|/|C_K(\sigma)|$, strictly smaller whenever σ commutes with a nontrivial within-block relabelling, and attained exactly then by Corollary 5.3. Three quantities stack in a fixed order in every configuration, $\log_2 |R_1| \geq \log_2 |K| = \log_2 |R_3| \geq \log_2 |G|$, with the first gap closing under alignment and the second under a trivial centraliser.

6.3 The $\mathbb{Z}/12$ instrument, read completely

The three transformations $+1$, $+2$ and $+3$ on $\mathbb{Z}/12$ recur through this paper and their full behaviour is worth setting out once, because they exhibit every phenomenon separately.

Their orbit structures are one block of twelve, two blocks of six and three blocks of four, so passive observation leaves 25.250, 13.814 and 7.755 bits respectively — the transitive case being by far the worst observed and the finest-orbit case the best. Under single-step intervention the ordering reverses: at any fixed resolution all three have the identical floor, 18.984 bits at two labels down to 6.000 at six, because the floor is a property of the labels alone. So $+3$ loses by moving from watching to intervening at coarse resolution, and $+1$ gains, from the same protocol change.

The alignment spread is largest in the middle of the resolution range. At three labels the best partition sits at 13.755 bits and the worst at 24.510, a spread of $2^{10.755}$, and every aligned partition sits at the floor while the median partition sits nine bits above it. At six labels the spread collapses to 2^6 because there is little room left for a partition to be misaligned.

Under the joint reader the three separate again. For $\langle +3 \rangle$ the passive reader is already sharp and the aligned intervening reader is highly redundant against it, delivering 23% of the weight an independence baseline would predict; for $\langle +1 \rangle$ the same comparison gives 28%, but from a far worse starting point. And in every case the aligned configuration produces the smallest final candidate set despite extracting the least marginal information — the inversion of rankings described in Section 8.

6A. The sampling horizon

The three intervention regimes were introduced as different protocols, but two of them are the same protocol read at different lengths, and saying so turns the hierarchy into a statement about sampling. For T at least 1, define the horizon- T reader as the one whose record is the multiset of label itineraries of length T , and write $R^{\wedge}(T)$ for its consistent set. Then R_3 is the limit of $R^{\wedge}(T)$, and the following hold in all $2,672$ configurations at $n = 4, 5, 6$, with no exceptions.

$R^{\wedge}(1) = \text{Sym}(X)$. One sample per state is the multiset of labels, which every permutation produces, so a single sample carries nothing whatever about the transformation.

$R^{\wedge}(2) = K \text{ sigma } K$, exactly. Two samples per state is the multiset of label pairs (label of x , label of $\text{sigma } x$), which is precisely the contingency table $N(P, \text{sigma})$. The single-step reader of Section 3 is therefore the two-sample reader, and the double coset is a two-gram statistic. Nothing about R_1 required a special protocol; it is what a trajectory reader sees in a window of length two.

$R^{\wedge}(T)$ is monotonically decreasing in T and saturates at $|K|$, by Theorem 5.1.

The critical horizon.

Let T^* be the least T at which $R^{\wedge}(T)$ equals its limit — the point past which further sampling is provably worthless. Exhaustively over every configuration at $n = 4, 5, 6$, the maximum of T^* over readers with m blocks is

$$\max T^* = n - m + 2 \text{ for } m \geq 2, \text{ and } T^* = 1 \text{ for } m = 1,$$

attained, and never exceeded. The distribution at $n = 6$ runs $T^* = 1$ in 11 configurations, 2 in 384, 3 in 1,234, 4 in 476, 5 in 117 and 6 in 11; the overall maximum is n .

blocks m	$n = 4$	$n = 5$	$n = 6$	$n - m + 2$
2	4	5	6	n
3	3	4	5	$n - 1$
4	2	3	4	$n - 2$
5	—	2	3	$n - 3$
6	—	—	2	$n - 4$

The two endpoints are worth reading directly. A fully labelled reader has $m = n$ and needs exactly two samples: with distinct labels the pair (label of x , label of σx) is σ itself, which is Corollary 3.4 at the discrete partition. A two-block reader needs n samples, the longest horizon any reader ever needs. Every additional label buys back exactly one step of required horizon, which is a linear exchange between resolution and time and is the sharpest form in this model of the statement that a coarse instrument must watch longer.

6A.1 What this says about sampling

The classical sampling theorem states that a band-limited signal is reconstructible from samples taken above twice its highest frequency. The hypothesis doing the work there is that the quantity of interest is already present in the sampled channel. It says nothing about structure lying outside the measurement map, and the model here makes that limitation exact rather than cautionary.

Two quantities separate. The horizon T^* controls how long a reader must sample before it has everything its channel can deliver, and it is finite, small, and given by the formula above. The floor $|K|$ controls what the channel can deliver at all, and by Theorem 5.1 it depends only on the partition — not on the transformation, not on the horizon, not on the number of repetitions. Sampling below T^* leaves information on the table; sampling above T^* is exactly zero information, not diminishing returns. The two failures are qualitatively different and no amount of the second fixes the first.

So a reader that samples forever still returns an equivalence class of size $|K|$, and the states inside a fibre of the projection are never separated by rereading that projection, however finely. What separates them is a different projection, which is the subject of Sections 9 through 11A: the complete binary family samples every two-block channel to saturation and still cannot resolve orientation, while two proper readers of the right shape recover σ exactly. Between those two results the operative variable is visibly the geometry of the reading rather than its quantity.

The finiteness of T^* has a counterpart in automata theory, where two states of a Moore machine with n states are distinguishable if and only if they are distinguishable within $n - 1$ steps. The bound here is of the same character and is stated for the whole candidate set rather than for a pair of states, with the resolution appearing explicitly in the rate.

7. Passive observation and the exchange rate

A passive reader recovers the cycle partition of σ . The transformations consistent with that observation are those whose cycle partition is the observed one, one cycle per block, of which there are $\prod_i (|B_i| - 1)!$.

This corrects a quantity used earlier in this programme. The expression $|\text{Aut}(\text{Inv}(G))| / |G|$ counts the group preserving the observed orbit partition and had been reported as the observational ambiguity. It is not the consistent set, which is not a group and was therefore not recognised as one:

```
<+1> orbits [12]  group-based 39,916,800 (25.250 b) consistent 39,916,800 (25.250 b) 1.0x
<+2> orbits [6,6]  group-based  86,400 (16.399 b) consistent  14,400 (13.814 b) 6.0x
<+3> orbits [4,4,4] group-based  3,456 (11.755 b) consistent   216 ( 7.755 b) 16.0x
```

The two agree in the transitive case only because $12!/12 = 11!$, an accident of that configuration from which the earlier figure had been generalised.

Comparing passive observation with aligned single-step intervention when the label partition is the orbit partition — the condition matters, since otherwise the counts range over different partitions and are not comparable — gives

$$\prod_i |B_i|! / \prod_i (|B_i| - 1)! = \prod_i |B_i|,$$

so single-step intervention pays $\sum_i \log_2 |B_i|$ bits of extra ambiguity for its block resolution. The mechanism is concrete rather than information-theoretic: watching delivers the cyclic order within each orbit for free, and fresh-draw preparation destroys it. For $\langle +3 \rangle$ on $\mathbb{Z}/12$ the penalty is exactly 6 bits, which is why coarse intervention there loses to doing nothing — passive observation leaves 7.755 bits while intervention needs six labels before its best case falls below that. Section 5 gives the honest qualification: the penalty is largely an artifact of the single-step protocol and shrinks under iteration.

8. Two readers, and a null model that fails

Combining a coarse intervening reader with a passive one is the natural next question. Write $M(\sigma) = K\sigma K \cap C_\lambda$, with C_λ the conjugacy class of the observed cycle type. M is never empty, since σ lies in both sets by construction; incompatibility between the two readings diagnoses a broken apparatus rather than an outcome.

Against the baseline $\Delta_{\text{indep}} = \log_2(n!/|C_\lambda|)$ — the reduction expected if $K\sigma K$ sat in $\text{Sym}(X)$ like a uniformly positioned subset of its size — a minority of configurations appear to gain more than independently.

n	configurations	configurations that gain	max excess (bits)	aligned P among the gainers
4	75	2	+0.5850	0
5	364	21	+1.0000	0
6	2,233	148	+1.5850	0

7	13,155	929	+2.3219	0
---	--------	-----	---------	---

Zero of 15,827 configurations across four group sizes have an aligned partition among the gainers. Alignment is the configuration in which single-step intervention is strongest and simultaneously the one in which trajectory information is most redundant, because both readers are constrained by the same orbit structure and the same structure cannot be counted twice. This is not alignment predicting uselessness: the rankings invert, and the aligned reader, while extracting the least marginal information from the second reader, still arrives at the smallest final candidate set — on $\mathbb{Z}/12$ with two labels, 16 aligned against 64 misaligned for $\langle +3 \rangle$, and 32 against 448 to 576 for $\langle +1 \rangle$. Alignment predicts redundancy, not usefulness.

8.1 The excess is unbounded and the baseline is wrong

Take $\lambda = (4, 1^{n-4})$ and $P = \{1,3\} \mid \text{rest}$, so that the 4-cycle carries the two-block to $\{0,2\}$, disjoint from itself — maximally misaligned in the precise sense that block and image share nothing. In table language the reader's 2×2 table is as far from block-diagonal as the margins permit. The meet splits as $\{1,3\} \mid \{0,2\} \mid \text{rest}$, giving

$$|K\sigma K| = (n-2)!^2 / (n-4)!, \quad |M| = (n-2)(n-3), \quad \text{apparent gain} = (n-2)(n-3)/4.$$

n	$ K\sigma K $ predicted	observed	$ M $ predicted	observed	excess (bits)
6	288	288	12	12	+1.5850
7	2,400	2,400	20	20	+2.3219
8	21,600	21,600	30	30	+2.9069
9	211,680	211,680	42	42	+3.3923

The $n = 9$ row was a prediction before it was a measurement. The excess is $\log_2((n-2)(n-3)/4)$ and grows without bound, settling the interpretation: the quantity measures double-coset geometry — imposed by K and its meet with the conjugated subgroup — and has nothing to do with how two readers combine. The baseline is retired rather than corrected.

Two intermediate claims are recorded as failures because the route to this statement runs through them. An apparent ceiling of exactly one bit on the excess held across 812 configurations of S_6 under four transformations; it was an artifact of testing four transformations rather than all cycle types, and the complete S_6 census gives 1.585 bits. Separately, parity was the obvious mechanism, since C_λ carries a definite sign and $K\sigma K$ splits evenly by sign whenever a block has size at least two; the correction cancels algebraically, $(|K|/2) \cdot |C_\lambda| / (n!/2) = |K| \cdot |C_\lambda| / n!$, and contributes nothing. Both are killed. The general lesson is portable: a combined candidate space is not the independent product of the individual candidate spaces, and a claim of super-additive measurement must first establish that its null respects the geometry of the candidate space.

9. Families of readers: what binary access recovers

Everything to this point concerns one reader. A family with differently shaped labels is the more realistic instrument, and it changes the conclusions in both directions.

Theorem 9.1 (binary access sees the undirected graph).

Let the family consist of every two-block partition of X — all $2^{n-1} - 1$ of them. Then the joint R_1 candidate set is exactly the set of permutations whose underlying undirected functional graph equals that of σ .

Proof. A two-block partition is determined by a subset $A \subseteq X$, and by Corollary 3.4 its table is determined by the single entry $f(A) = |\{x \in A : \sigma x \in A\}|$ together with the margins. So the family's total record is the function $A \mapsto f(A)$ over all subsets. Now $f(A)$ counts the directed edges of σ with both endpoints in A , and a directed edge $x \rightarrow \sigma x$ lies inside A exactly when the undirected edge $\{x, \sigma x\}$ does. Hence f depends only on the undirected graph, and any permutation with the same undirected graph produces the same record. Conversely the record determines the graph: taking $A = \{x\}$ gives the fixed points, and for non-fixed x, y the value $f(\{x, y\}) - [\sigma x = x] - [\sigma y = y]$ equals $[\sigma x = y] + [\sigma y = x]$, which detects the undirected edge. ■

Corollary 9.2 (the orientation count).

The joint binary candidate set has size 2^c , where c is the number of cycles of σ of length at least three. The undirected graph of a permutation is a disjoint union of cycles of length ≥ 3 , edges (2-cycles) and loops (fixed points); a cycle of length ≥ 3 admits exactly two orientations that are single cycles, while 2-cycles and fixed points admit one. Cycle orientations are chosen independently, since f is orientation-blind on each.

Confirmed on every cycle type at $n = 4, 5, 6, 7$ — 38 cycle types in total, with the candidate set equal to the orientation set in every case. A representative slice:

n	cycle type λ	cycles of length ≥ 3	predicted 2^c	observed $ N $	set equals orientations
6	(6)	1	2	2	yes
6	(3,3)	2	4	4	yes
6	(4,2)	1	2	2	yes
6	(2,2,2)	0	1	1	yes
7	(4,3)	2	4	4	yes
7	(3,3,1)	2	4	4	yes
7	(2,2,2,1)	0	1	1	yes

The result upgrades an earlier statement of this material, where the case of a single n -cycle had been verified at three values of n and offered as a conjecture with a mechanism sketch. The mechanism is Corollary 4.3, the proof is four lines, and the correct generality is all cycle types with an exact count.

10. The handedness threshold

Corollary 9.2 says the last thing binary access loses is orientation. Section 5 says trajectory access is usually redundant against iterated access. Together they predict that orientation is where trajectory access should matter if it matters anywhere, and it is.

10.1 Iteration does not help

Proposition 10.1.

For any binary partition P and any t , the table $N(P, \sigma^t)$ is symmetric. Hence σ^{-1} produces the same R_2 record as σ for every binary reader, and no family of binary readers can distinguish a transformation from its inverse under iterated access.

This is Corollary 4.3 applied to each power: the margins of $N(P, \sigma^t)$ are the block sizes for every t , and a 2×2 table with equal margins is symmetric. Confirmed exhaustively over every power and every binary reader at $n = 5, 6, 7$.

10.2 Trajectory access, and when it helps

Under R_3 the situation changes, and the criterion is combinatorial. Restrict attention to σ a single k -cycle and let a binary reader assign labels around the cycle, producing a cyclic binary word w of length k . The R_3 record is the multiset of label itineraries, which for windows of length at least k determines w up to rotation — that is, the record is the necklace of w . The inverse traverses the cycle backwards, so its record is the necklace of the reversed word.

Theorem 10.2 (handedness threshold).

A binary reader distinguishes a k -cycle from its inverse under R_3 if and only if its label word is a chiral necklace, one that differs from its reversal. Such a word of length k exists if and only if $k \geq 6$. Consequently the complete binary family recovers a k -cycle exactly under R_3 for $k \geq 6$, and leaves the pair $\{\sigma, \sigma^{-1}\}$ for $k \leq 5$, at every protocol in the hierarchy.

The threshold is the classical one. Counting binary necklaces against binary bracelets — the latter identifying a word with its reversal — the two agree up to length five and first differ at six:

k	binary necklaces	binary bracelets	chiral pairs	chiral word exists	R_3 binary family recovers a k -cycle
3	4	4	0	no	$\{\sigma, \sigma^{-1}\}$
4	6	6	0	no	$\{\sigma, \sigma^{-1}\}$
5	8	8	0	no	$\{\sigma, \sigma^{-1}\}$
6	14	13	1	yes	$\{\sigma\}$
7	20	18	2	yes	$\{\sigma\}$
8	36	30	6	yes	(predicted $\{\sigma\}$)
9	60	46	14	yes	(predicted $\{\sigma\}$)

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

The computed recovery column at $k = 3$ through 7 was produced independently of the necklace counts, by intersecting the R_3 records of the complete binary family over all of $\text{Sym}(X)$, and it matches the chirality column exactly. The shortest chiral binary necklace is the classical example of length six.

Three consequences are worth separating.

Handedness is not a matter of how much data a reader collects. A binary reader with unlimited repetitions of a 5-cycle never resolves its orientation, because the obstruction is a symmetry of its record rather than a shortage of samples. Adding readers does not help either, since every binary reader is already in the family.

Handedness is not a matter of resolution alone. Binary resolution suffices at $k \geq 6$ and fails at $k \leq 5$, with the same number of labels in both cases. What changes is whether a chiral word of that length exists at all — a property of the length, not of the reader's diligence.

Handedness is exactly where protocol matters. R_1 and R_2 never see it; R_3 sees it whenever the arithmetic permits. This is the one place in the census where trajectory access is not redundant against iteration, and Section 5 shows it is a small place: 5.9% of configurations at $n = 6$. The rare cases are the informative ones.

10.3 What this settles in the wider programme

This programme has carried a standing hypothesis, logged under several domain lenses, that a small detune reveals handedness — that structured inputs respond coherently to a mirror perturbation while null baselines stay flat. Theorem 10.2 makes that hypothesis decidable in this model rather than merely testable.

Handedness is a property of the orientation of the transformation's functional graph. It is invisible to any single-step or iterated reader at binary resolution, invisible to trajectory readers whose label word is achiral, and visible otherwise. An instrument that reads two levels and does not track individual trajectories will therefore report a flat response to a mirror perturbation regardless of what is there, and the flat response is a fact about the instrument.

11. Exact recovery is a property of families

A single reader recovers σ exactly only when it is fully labelled: $R_1(\sigma, P) = \{\sigma\}$ forces $K = \{e\}$, since $G \subseteq R_1$ and $|G| = 1$ requires K to centralise σ , which for a Young subgroup with any block of size at least two is impossible unless the reader is discrete. A family behaves differently.

Result 11.1.

For every cycle type at $n = 5$ and $n = 6$, two proper partitions suffice for exact recovery under R_1 , and no single proper partition ever does. Exhaustively over all 50 proper partitions at $n = 5$ and all 201 at $n = 6$, for all 7 and 11 cycle types respectively: the minimum family size is 2 in every case, and the count of single readers achieving exact recovery is 0 in every case. Witnesses:

$n = 5$: $\{\{0\}, \{1\}, \{2,3,4\}\}$ and $\{\{0,2\}, \{3\}, \{1,4\}\}$
 $n = 6$: $\{\{0\}, \{1\}, \{2,3,4,5\}\}$ and $\{\{0,2\}, \{3\}, \{4\}, \{1,5\}\}$

Neither witness is fully labelled; each has a block of size two or more and hence a nontrivial gauge. The two gauges do not commute with one another's structure, and each reader breaks the other's. The transition from 1 to 2 is therefore not a matter of accumulating resolution but of the family ceasing to have a common gauge.

11.2 How common are sufficient pairs

Two readers suffice, but not any two. Counting over all pairs of proper partitions:

n	cycle type	pairs recovering exactly	of all pairs	rate
5	(5)	180	1,225	14.7%
5	(3,2)	210	1,225	17.1%
5	(2,1,1,1)	363	1,225	29.6%
5	identity	525	1,225	42.9%
6	(6)	2,390	20,100	11.9%
6	(3,3)	2,461	20,100	12.2%
6	(2,2,2)	4,586	20,100	22.8%
6	identity	7,360	20,100	36.6%

Sufficient pairs are common but not generic, and the rate falls monotonically as the transformation acquires longer cycles: a 6-cycle is the hardest object to pin at $n = 6$ and the identity the easiest, with a factor of three between them. The ordering is intuitive in table language — a long cycle spreads its mass across the table for most cuts, and spread tables are the ones whose double cosets are large.

This withdraws a claim made in an earlier draft of this material and repeated in the programme's notes: that exact recovery occurs only at full label resolution. It is true of one reader and false of a family, and the false version is the one that had been generalised.

Note the contrast with Section 10. Two well-chosen proper readers recover any transformation exactly, yet the entire binary family — $2^{n-1} - 1$ readers — cannot resolve one bit of orientation. Reader count is not the operative variable; the shape of the family is. A family closed under an operation that fixes the record while moving the transformation can never separate what that operation identifies, however large it is, and the binary family is closed under exactly one such operation.

11A. Binary is the only failing resolution

Section 9 shows the complete two-block family stops at the undirected graph. The obvious question is whether the obstruction is a feature of coarse access in general or of binary access specifically, and the answer is sharp.

Result 11A.1.

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828
 Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

For every $m \geq 3$, the complete family of all m -block partitions recovers σ exactly under R_1 , for every cycle type. Verified exhaustively at $n = 5$ with $m = 3$ (25 readers) and $m = 4$ (10 readers), and at $n = 6$ with $m = 3$ (90 readers) and $m = 4$ (65 readers). By contrast the complete two-block family leaves 2^c candidates, with c the number of cycles of length at least three, exactly as Corollary 9.2 predicts:

n	blocks per reader	readers in family	exact for every cycle type
5	2	15	no — leaves 2^c
5	3	25	yes
5	4	10	yes
6	2	31	no — leaves 2^c
6	3	90	yes
6	4	65	yes

So the handedness obstruction is not about coarseness. A three-block reader is barely finer than a two-block one, and the family of them is smaller in the $n = 5$ case than the binary family it outperforms. What fails at $m = 2$ is that a table with two rows and equal margins is forced symmetric, and symmetry of the table is exactly indistinguishability from the inverse. At $m = 3$ the constraint disappears: tables with three rows and equal margins need not be symmetric, and asymmetric ones exist for every cycle type tested.

This gives the cleanest statement of the phenomenon. Orientation-blindness is a property of the number two, entering through the margins of a 2×2 table, and it is invisible to any argument phrased in terms of how much resolution a reader has.

11B. A reader that runs

Every reader so far has been a static partition. The source runs and the reader does not, which makes the model an asymmetric one: a moving thing observed by a still frame. Nothing forces that. Let the reader carry its own permutation π and rotate its labels as it goes, so that at time s it reads through the partition $\pi^s P$, and the symbol it records for a state x is the label of π^s applied to $\tau^s x$. Call this a co-moving reader. It is specified by a pair (P, π) and it is given nothing extra: the same two labels, the same finite horizon, and π is its own state rather than a measurement of the source.

Result 11B.1 (co-motion is not extra resolution).

Every frame a co-moving binary reader uses is a two-block partition, so the set of partitions it consults is contained in the complete static binary family. By Theorem 9.1 that family, all $2^{(n-1)-1}$ of its members read to saturation, leaves exactly the orientations of the undirected functional graph. The static family therefore has strictly more partitions available than any co-moving reader uses, and strictly less power. Whatever the co-moving reader gains is not resolution and not partition count. It is the temporal register between its frames and the transformation.

Result 11B.2 (one moving reader beats every static one).

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828
 Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)
github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

For every non-identity cycle type at $n = 4$ and $n = 5$, a single co-moving binary reader recovers sigma exactly. The complete static binary family leaves 2^c candidates for the same transformations, where c counts the cycles of length at least three.

n	cycle type	complete static binary family	best single co-moving binary reader
4	(4)	2	1
4	(3,1)	2	1
4	(2,2)	1	1
5	(5)	2	1
5	(4,1)	2	1
5	(3,2)	2	1
5	(3,1,1)	2	1
5	identity	1	2
4	identity	1	4

Orientation in particular is broken by a large majority of reader dynamics rather than by a rare choice: of all pairs (P, π) with P a two-block partition, the fraction separating a full cycle from its inverse is 66.7% at $n = 3$, 57.1% at $n = 4$, 83.9% at $n = 5$ and 85.5% at $n = 6$. Recall that no static binary reader separates them at any n , at any horizon, after any number of repetitions.

Result 11B.3 (a moving reader cannot see a still source).

The last two rows of the table are the exception and they are not a defect of the construction. When sigma is the identity the co-moving reader is strictly worse than the static family, leaving four candidates at $n = 4$ and two at $n = 5$ where a static reader leaves one. The reason is immediate from the definition: the recorded symbol is the label of $\pi^{-s} \tau^x$, so a reader in motion cannot attribute the change it sees to the source rather than to itself. Against a still source its own motion is pure confusion. Only relative motion is legible, and a reader that moves has given up the ability to certify that nothing moved.

This is the sharpest statement the model offers about what reading is. Resolution, horizon and reader count are all quantities, and Sections 6A, 9 and 11 show each of them saturating: labels set a floor that no sampling penetrates, horizons stop paying after $n - m + 2$ steps, and a complete family of the wrong shape stalls one bit short forever. Co-motion is not a quantity. It is a relation between two running things, it is the only variable in this paper that breaks the orientation obstruction from a single two-label reader, and it comes with a cost that the static reader does not pay.

What fails is not characterised. For a fixed partition at $n = 5$, sixteen of the 120 reader dynamics fail to separate a cycle from its inverse, and that failing set is neither the cyclic group generated by sigma nor the centraliser of sigma. It is recorded as open.

12. What this says about inferring symmetry

The programme's position has been that a symmetry cannot be observed, only failed to be broken, so every claimed symmetry is a lower bound on the true one and the systematic error runs toward attributing the reader's own structure to the source. The model here makes that statement exact and, in one respect, sharper.

A reader's failed attempts to break a symmetry return the equivalence class its access admits, not a group. Under R_1 that class is a double coset, which is not a group; under R_3 it lies between the gauge and the double coset; under passive observation it is a set of permutations with a given cycle partition, also not a group. Naming a group as the recovered object is a second-order instance of the same error the programme already names — reporting the reader's own preferred structure in place of what the data determines.

The direction of the error is confirmed and quantified. A single reader's residual always contains the gauge, and the gauge is exactly the reader's own relabelling freedom. What the census adds is that the residual is usually strictly larger than the gauge — 46% of configurations at $n = 6$ for the strongest coarse reader — so attributing the entire residual to one's own labels is also wrong, in the other direction.

And the binary family gives the cleanest available example of an unbreakable symmetry that is not a symmetry of the object. Orientation reversal is not a symmetry of σ ; σ and σ^{-1} are different transformations. It is a symmetry of the access structure, and no amount of failed breaking within that structure will distinguish them. The lesson transfers directly: before concluding that two configurations are identical because no experiment separates them, check whether the family of experiments is closed under an operation that exchanges them.

13. Status

The grades below name the scope of each claim, not the method that established it. An exhaustive enumeration over a finite domain, with no sampling, proves the proposition over that domain; rewriting it symbolically adds nothing. ALL n marks a claim that holds for every finite n , which requires an argument no enumeration can supply. A bound such as $n \leq 6$ ONLY marks a claim established exhaustively over the configurations at those sizes and asserted nowhere else.

This convention exists because of how the claims in this paper actually failed. Four were withdrawn during the work, and every one of them came from code that ran correctly and enumerated exactly what it reported. The closure gap generalised from a configuration where $12!/12 = 11!$ is exactly true. The one-bit ceiling held across 812 configurations of S_6 and was false because that census covered four transformations rather than fifteen cycle types. The Case A and Case B split was exhaustive at $n \leq 6$ and was superseded by a simpler identity. The containment step in Theorem 5.5 was first written with the word-classes of sigma instead of tau and fails in 478 configurations at $n = 6$. In each case the run was sound and the quantifier reached past the domain the run covered — which is the same way a symbolic argument fails when its hypotheses are wider than its reasoning.

A census is itself a reader, and Section 12 applies to it. Its resolution is the set of configurations it enumerates; everything outside sits in its gauge; and what it returns is the equivalence class its access admits rather than the object. The four failures above are one error, reading a census residual as the

transformation, which is why widening the domain rather than reasoning harder is what corrected each of them.

Claim	Grade	Basis
R_1 record = contingency table $N(P, \sigma)$	ALL n	Proposition 3.2, from the definition
R_1 consistent set = $K\sigma K$	ALL n (classical)	parabolic double coset $\leftrightarrow$ table; checked $n = 6$
$ K\sigma K = (\prod B_i !)^2 / \prod N_{ij} !$	ALL n	argument + exhaustive at $n = 4, 5$ (6,600 pairs)
$\sigma^{-1} \in K\sigma K$ iff $N(P, \sigma)$ symmetric	ALL n	argument + exhaustive at $n \leq 6$ (152,760 triples)
$ P \leq 2 \Rightarrow$ table symmetric	ALL n	equal margins force it; exhaustive at $n \leq 6$
Binary family recovers the undirected graph	ALL n	Theorem 9.1; exhaustive at $n \leq 7$, all cycle types
Binary candidate count = 2^c	ALL n	c = cycles of length ≥ 3 ; exhaustive at $n \leq 7$
Iteration cannot break orientation	ALL n	margins argument; exhaustive at $n = 5, 6, 7$
R_3 breaks orientation iff label word is chiral	$k \leq 7$ ONLY	exhaustive at $k = 3-7$; necklace step unproved for general k
Chiral binary word exists iff $k \geq 6$	ALL k (classical)	necklaces vs bracelets first differ at 6
Two proper readers suffice, one never does	$n \leq 6$ ONLY	exhaustive, all cycle types; no construction for general n
One co-moving binary reader recovers σ exactly	$n \leq 5$ ONLY	every non-identity cycle type; static family leaves 2^c
Co-moving reader is worse against the identity	$n \leq 5$ ONLY	4 and 2 candidates vs 1; own motion is not attributable
$G \subseteq R_3 \subseteq R_2 \subseteq R_1$, $ G = K / C_K(\sigma) $	ALL n	K -conjugation preserves every record
$R^{(1)} = \text{Sym}(X)$; $R^{(2)} = K\sigma K$ exactly	ALL n	the contingency table is a 2-gram statistic; exhaustive $n \leq 6$
$R^{(T)}$ monotone, saturating at $ K $	ALL n	Theorem 5.1; exhaustive $n \leq 6$
max critical horizon $T^* = n - m + 2$ ($m \geq 2$)	$n \leq 6$ ONLY	exhaustive, attained and never exceeded; no argument for general n
Single-step restriction carries most ambiguity	$n \leq 6$ ONLY	$ R_2 < R_1 $ in 1,838 of 2,233 at $n = 6$; a rate, not a law
Trajectory adds little beyond iteration	$n \leq 6$ ONLY	$ R_3 < R_2 $ in 132 of 2,233 at $n = 6$; a rate, not a law
$\log_2 K\sigma K \geq \log_2 K $, equality iff σ normalises K	ALL n	normaliser argument; $\mathbb{Z}/12$ sweep, four resolutions
Passive consistent set = $\prod (B_i - 1)!$	ALL n	cycle count; earlier group-based figure overstated 6 $\times$ and 16 $\times$
Aligned P never among the gainers	$n \leq 7$ ONLY	exhaustive, 0 of 15,827; no argument for general n

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

Extremal gain = $(n-2)(n-3)/4$ on that family	ALL n	closed form from the meet; $n = 9$ out of sample
$ R_3 = K $ for every σ and P	ALL n	Theorem 5.1 and Theorem 5.5; exhaustive $n \leq 6$, four horizons, $n = 7$
$ G $ divides $ R_3 $	ALL n	Corollary 5.2, orbit–stabiliser
$R_3 = G$ iff $C_K(\sigma)$ trivial	ALL n	Corollary 5.3; accounts for the 54% exactly
aligned $\Leftrightarrow R_2 = R_1 \Leftrightarrow R_3 = R_1 \Leftrightarrow R_1 = K $	$n \leq 6$ ONLY	exhaustive, as sets, 2,672 configurations
R_3 classes $\leftrightarrow$ return-map conjugacy classes	ALL n	Theorem 5.5, holonomy argument; exhaustive $n \leq 6$
Equal class sizes iff every word-class ≤ 2	ALL n	Corollary 5.6; supersedes the Case A/B split
One-bit ceiling on the excess	FALSIFIED	held on an incomplete census of S_{6i} ; 1.585 complete, 3.392 at $n = 9$
Excess is a parity effect	FALSIFIED	correction cancels exactly
Independence baseline $\log_2(n!/ C_\lambda)$	RETIRED	measures coset geometry, not reader overlap
Exact recovery requires full labelling	WITHDRAWN	true for one reader, false for a family

14. What this does not establish

Theorem 10.2 is proved in the direction that matters and verified in the other. That a chiral label word distinguishes σ from σ^{-1} under R_3 follows from the record being the necklace; that no achiral word can is likewise immediate. The step taken on trust is that the R_3 record of a k -cycle is exactly the necklace of the label word, which is argued for window lengths at least k and confirmed at $k = 3$ through 7, not proved in general. Extension to permutations with several cycles under R_3 is not attempted here; only the R_1 and R_2 statements are proved at that generality.

Result 11.1 is exhaustive at $n = 5$ and $n = 6$ and is not a theorem. Whether two proper readers always suffice for every n is open, and the witnesses give no construction.

The extremal family of Section 8.1 is proved maximal only where the census is exhaustive, which is $n \leq 7$. At $n = 8$ the sweep covered two- and three-block profiles, and the $n = 9$ computation verifies the family rather than its global optimality.

Proposition 5.4 is verified exhaustively at $n \leq 6$ and is not proved. Three of its four equivalences are elementary; the content is that trajectory access adds nothing over single-step access exactly under alignment, and no argument is given for that direction.

The reader model assumes preparation by label does not control which state within a block is prepared. A reader with partial state control interpolates between R_3 and full labelling, and that regime is not measured; nothing here bounds what such a reader recovers.

All statements concern finite permutation actions on finite sets. No continuum, measure-theoretic or asymptotic claim is made or implied. The double-coset, contingency-table and necklace machinery is classical and cited as such; the contribution is the identification of the reader's object with it, the inversion

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

criterion that follows, the handedness threshold, and the separation of protocol from resolution from structure.

14A. Related work

Three classical bodies of work are used and none is extended.

Parabolic double cosets and contingency tables. The parametrisation of double cosets of Young subgroups of the symmetric group by two-way tables with prescribed row and column sums is due to Diaconis and Gangolli, with the underlying bijection attributed to Bergeron, and is the basis of Section 3. Enumeration of parabolic double cosets, including lex-minimal presentations and counts in Coxeter groups generally, has been developed by Billey, Konvalinka, Petersen, Slofstra and Tenner and others; the present paper uses only the parametrisation and the elementary size formula. The identity $|HgK| = |H||K|/|H \cap gKg^{-1}|$ is textbook, as is the fact that the intersection of two Young subgroups is the Young subgroup of the meet of the underlying partitions.

Lumping and identifiability. Observing a dynamics through a partition of its state space is lumping, and the question of what can be identified from lumped observations has a long literature in Markov chain theory and in the statistical-physics work on coarse-grained dynamics and time-reversal estimation. The present model is the deterministic, finite, permutation case of that setting, where the answers are exact rather than asymptotic. The distinction drawn here between iterated fresh-draw access and trajectory access corresponds to the difference between observing the lumped one-step statistics of all powers and observing lumped sample paths.

Necklaces and bracelets. The count of binary necklaces and of binary bracelets, and the fact that they first differ at length six, are classical enumerative results. Section 10 uses only the resulting chirality threshold. That a length-six binary word is the shortest that differs from its reversal up to rotation is the entire content of the handedness threshold once the record has been identified with a necklace.

What is not in that literature, as far as the author is aware, is the identification of a reader's recovered object with the double coset, the resulting inversion criterion, the consequence that binary access is exactly orientation-blind while three-block access is not, and the separation of a reader's residual ambiguity into protocol, gauge and structural components with measured proportions.

15. Reproduction

Every number in this paper is produced by the companion script, which runs the operational identification of Section 3, the product formula, the inversion criterion over all 152,760 triples, the ladder census of Section 5, the $\mathbb{Z}/12$ floor sweep, the observation comparison, the exhaustive S_4 – S_7 census and extremal prediction of Section 8, the binary-family theorem across all cycle types, the necklace-chirality comparison of Section 10, the minimum-family and pair-density results of Section 11, and the class-size dichotomy of Proposition 5.1. No randomness is used; every census stated as exhaustive enumerates its full domain. Total runtime is a few minutes on one core.

16. Open problems

Seven questions are left open, stated precisely enough to be attacked. Three earlier ones have been removed: the divisibility of the trajectory residual by the gauge, now Corollary 5.2; the characterisation of readers attaining their gauge, now Corollary 5.3; and the decomposition of the trajectory residual into classes, now Theorem 5.5.

Problem 1. Prove that the R_3 record of a k -cycle at binary resolution is exactly the necklace of the label word, for all k . The statement is used in Theorem 10.2 and verified at $k = 3$ through 7. A proof would upgrade the handedness threshold from verified to proved at full generality.

Problem 2. Extend the handedness threshold to permutations with several cycles under R_3 . The R_1 and R_2 statements are proved at that generality by Corollary 4.3, and Corollary 9.2 gives the R_1 count as 2^c ; what replaces 2^c under trajectory access, when the cycles have different lengths and the label word restricted to each may be independently chiral or achiral, is not determined here.

Problem 3. Prove or refute that two proper partitions always suffice for exact recovery under R_1 , for every n and every σ . Verified exhaustively at $n = 5$ and $n = 6$. A construction would be more useful than an existence proof, since the witnesses found here are not obviously a pattern.

Problem 4. Prove Proposition 5.4, specifically that $R_3 = R_1$ exactly when P is aligned with σ . Theorem 5.1 fixes $|R_3| = |K|$ and Section 6.1 fixes $|R_1| \geq |K|$ with equality under alignment, so one direction is immediate; what is unproved is that misalignment always leaves some transformation consistent at one step but not along trajectories.

Problem 5. Determine whether $(n-2)(n-3)/4$ is the global maximum of the Section 8.1 quantity for every n . Exhaustive at $n \leq 7$; the family is verified but not shown extremal beyond that.

Problem 6. Prove the critical-horizon law, that a reader with m blocks needs at most $n - m + 2$ samples and that the bound is attained. It is exhaustive at $n \leq 6$ and has the shape of a Moore-style distinguishability bound with the resolution appearing in the rate, which suggests the automata-theoretic argument may transfer.

Problem 7. Characterise the reader dynamics that fail. For a fixed two-block partition at $n = 5$, sixteen of 120 choices of π leave a cycle indistinguishable from its inverse, and the failing set is neither the group generated by σ nor its centraliser. A criterion would say when co-motion is legible, and the identity case of Result 11B.3 suggests the answer involves the relation between π and σ rather than either alone.

```

1. """Verification script for Reader Surplus Section X.
Reproduces every number in the section. Runtime ~35s through n=7, ~40s including n=9 prediction.
python3 verify_section_X.py
""" import itertools, math, time from math import factorial, log2 from itertools import permutations
from collections import Counter, deque

def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue          l=0; j=i          while not seen[j]: seen[j]=True; j=p[j]; l+=1
    c.append(l)
    return tuple(sorted(c,reverse=True))
def class_size(lam,n):
    cnt=Counter(lam); d=1          for l,m in cnt.items(): d*=(l**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for combo in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,combo):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcoset(s,gens):
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def setparts(el):
    el=list(el)
    if not el: yield []; return

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

    f,rest=el[0],el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
        yield [(f,)]+r
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in itertools.combinations(el[1:],size-1):
        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))]+t

print("X.2 observation: consistent set vs Aut(Inv)/|G| on Z/12")
for k in (1,2,3):
    n=12; G=n//math.gcd(n,k); sizes=[G]*(n//G)
    a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}>: Aut-based {a:>9} ({log2(a):6.3f} b) consistent {h:>9} ({log2(h):6.3f} b)
ratio {a/h:.1f}x")

print("\nX.3 record-consistent set equals the double coset (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L
def record(t,L,P): return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
for P,k in [([(0,1,2),(3,4,5)],1),([(0,3),(1,4),(2,5)],1),([(0,1),(2,3),(4,5)],2),
            ([[(0,1,2),(3,4,5)],3),([(0,2,4),(1,3,5)],1)]:
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record(t,L,P)==record(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    print(f"    P={P} +{k}: |consistent|={len(cons):4d} |Ksk|={len(I):4d}
match={len(cons)==len(I)}")

print("\nX.4 floor log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[]
        for P in uparts(range(N),bs):
            amb=young(P)**2//young(meet(P,imgP(P,s)))
            rows.append((log2(amb), aligned(P,s)))
        rows.sort()
        K=log2(young([tuple(range(bs))* (N//bs)]))
        inv=[r[0] for r in rows if r[1]]
        ok = abs(rows[0][0]-K)<1e-9 and min(inv)==max(inv)==rows[0][0]
        print(f"    +{k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f}
median={rows[len(rows)//2][0]:7.3f}"
            f"    worst={rows[-1][0]:7.3f} aligned-attains-floor={ok}")

print("\nX.5/X.6 exhaustive census S_4..S_7: aligned P never gains over the independence baseline")
for n in (4,5,6,7):
    t0=time.time(); allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9

```

```

for lam in parts_of(n):
    s=rep_of(lam,n); Cl=class_size(lam,n); d_ind=log2(factorial(n)/Cl)
    for P in allP:
        I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
        e=log2(len(I))-log2(M)-d_ind; tot+=1
        if e>1e-9:
            sup+=1
            if aligned(P,s): alsup+=1
            mx=max(mx,e)
    print(f"    n={n}: {tot:6d} configs, {sup:4d} gain, max excess {mx:+.4f} bits, aligned&gain
{alsup} [{time.time()-t0:.0f}s]")

print("\nX.6 extremal family: lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"    n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):>7}    |M| {(n-2)*(n-3):>3}/{M:>3}"
          f"    gain {(n-2)*(n-3)/4:7.3f}/{R:7.3f}    excess {log2(R):+.4f} bits")

2. X.2 observation: consistent set vs Aut(Inv)/|G| on Z/12    <+1>: Aut-based 39916800 (25.250 b)
consistent 39916800 (25.250 b) ratio 1.0x    <+2>: Aut-based 86400 (16.399 b) consistent
14400 (13.814 b) ratio 6.0x    <+3>: Aut-based 3456 (11.755 b) consistent 216 ( 7.755
b) ratio 16.0x

X.3 record-consistent set equals the double coset (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] +1: |consistent|= 324 |KsK|= 324 match=True    P=[(0, 3), (1, 4), (2,
5)] +1: |consistent|= 8 |KsK|= 8 match=True    P=[(0, 1), (2, 3), (4, 5)] +2: |consistent|=
8 |KsK|= 8 match=True    P=[(0, 1, 2), (3, 4, 5)] +3: |consistent|= 36 |KsK|= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] +1: |consistent|= 36 |KsK|= 36 match=True

X.4 floor log2|K|, attained exactly on aligned partitions (Z/12)
+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains-
floor=True    +1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-
attains-floor=True    +1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680
aligned-attains-floor=True    +1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000
aligned-attains-floor=True    +2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628
aligned-attains-floor=True    +2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510
aligned-attains-floor=True    +2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680
aligned-attains-floor=True    +2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000
aligned-attains-floor=True    +3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628
aligned-attains-floor=True    +3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510
aligned-attains-floor=True    +3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680
aligned-attains-floor=True    +3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000
aligned-attains-floor=True

X.5/X.6 exhaustive census S_4..S_7: aligned P never gains over the independence baseline    n=4:
75 configs,    2 gain, max excess +0.5850 bits, aligned&gain 0 [0s]
n=5:    364 configs,    21 gain, max excess +1.0000 bits, aligned&gain 0 [0s]
n=6:    2233 configs,    148 gain, max excess +1.5850 bits, aligned&gain 0 [1s]
n=7:    13155 configs,    929 gain, max excess +2.3219 bits, aligned&gain 0 [38s]

X.6 extremal family: lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4    n=6: |KsK|    288/
288    |M| 12/ 12    gain 3.000/ 3.000    excess +1.5850 bits    n=7: |KsK|    2400/ 2400    |M|
20/ 20    gain 5.000/ 5.000    excess +2.3219 bits    n=8: |KsK|    21600/ 21600    |M| 30/ 30
gain 7.500/ 7.500    excess +2.9069 bits    n=9: |KsK|    211680/ 211680    |M| 42/ 42    gain
10.500/ 10.500    excess +3.3923 bits

3. """Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as
exhaustive enumerates its full domain. Runtime ~2 min on one core.
"""

4.

```

```

5. import itertools, time from math import factorial, log2 from itertools import permutations,
combinations from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)
    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(l**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))

# ----- partitions and Young subgroups -----
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcoiset(s, gens):
    """KsK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f,rest=el[0],el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
        yield [(f,)]+r
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:],size-1):
        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))+t
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L

# ----- reader records -----
def record_R1(t,L,P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        s=[]; y=x
        for _ in range(T): s.append(L[y]); y=t[y]
        out.append(tuple(s))
    return tuple(sorted(out))

T0=time.time()
print("=*74)
print("SECTION 3.  R1 consistent set = the double coset KsK   (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P,k in [([(0,1,2),(3,4,5)],1),([(0,3),(1,4),(2,5)],1),([(0,1),(2,3),(4,5)],2),
            ([[(0,1,2),(3,4,5)],3),([(0,2,4),(1,3,5)],1)]:
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record_R1(t,L,P)==record_R1(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    cf=young(P)**2//young(meet(P,imgP(P,s)))
    print(f"   P={P} sigma={k}: |consistent|={len(cons):4d}   |KsK|={len(I):4d}   "
          f"closed form={cf:4d}   match={len(cons)==len(I)==cf}")

print("\n"+="=*74)
print("SECTION 4.  the ladder  G <= R3 <= R2 <= R1   (class reps x all set partitions)")
print("   n   configs   |R2|<|R1|   |R3|<|R2|   R3==G   R1==G")
for n in (4,5,6):

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

allp=list(permutations(range(n))); allP=list(setparts(range(n)))
tot=s21=s32=eqG=r1G=0; mult=Counter()
for lam in parts_of(n):
    s=rep_of(lam,n); T=order(s)
    pw=[]; q=s
    for _ in range(T): pw.append(q); q=comp(s,q)
    for P in allP:
        K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
        R1=dcoset(s,gens)
        pc=[dcoset(p,gens) for p in pw]
        R2=set()
        for t in R1:
            q=t; ok=True
            for i in range(T):
                if q not in pc[i]: ok=False; break
                q=comp(t,q)
            if ok: R2.add(t)
        tgt=itinerary(s,L,2*n)
        R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
        G=set(comp(k,comp(s,inv(k))) for k in K)
        assert G<=R3<=R2<=R1
        tot+=1
        s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
        eqG+= R3==G; r1G+= R1==G
        mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
print(f"    {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
if n==6:
    print(f"        |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"="*74)
print("SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[(log2(young(P)**2//young(meet(P,imgP(P,s))))), aligned(P,s)) for P in
uparts(range(N),bs)]
        rows.sort(); K=log2(young([tuple(range(bs))]*(N//bs)))
        al=[r[0] for r in rows if r[1]]
        ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
        print(f"    sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
            f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"="*74)
print("SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b) "
        f"consistent {h:>9} ({log2(h):6.3f} b) ratio {a/h:.1f}x")
print(f"    transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7. joint reader census S_4..S_7, and the extremal family")
print("    n configs gain max excess aligned among gainers")
for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:

```

```

I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
e=log2(len(I))-log2(M)-d; tot+=1
if e>1e-9:
    sup+=1
    if aligned(P,s): alsup+=1
    mx=max(mx,e)
print(f"    {n} {tot:8d} {sup:6d}    {mx:+.4f}    {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"    n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
          f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

print("\n"+"="*74)
print("SECTION 8. families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"    n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
    three=[P for P in setparts(range(n)) if len(P)==3]
    kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    cos=[dcoset(s,blockgens(P,n)) for P in props]
    best=None
    for m in (1,2):
        for cb in combinations(range(len(props)),m):
            I=set(allp)
            for i in cb: I&=cos[i]
            if len(I)==1: best=(m,[props[i] for i in cb]); break
    if best: break
    print(f"    n={n}: 3-block readers breaking the tie: {kills}/{len(three)}; "
          f"minimum proper partitions for exact recovery = {best[0]}")
    print(f"    witness: {best[1]}")
print(f"\nall sections reproduced in {time.time()-T0:.0f}s")
6. =====
SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma=+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma=+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma=+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma=+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma=+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
=====
SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)
n configs |R2|<|R1| |R3|<|R2| R3==G R1==G 4 75 40 3
39 (52.0%) 5 5 364 262 18 201 (55.2%) 7 6 2233

```

```

1838      132      1205 (54.0%)      11      |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41,
4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72, 16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1} (no
non-integer ratios: True)

=====
SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)
sigma=+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
<+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
<+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio 6.0x
<+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 ( 7.755 b) ratio
16.0x transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
n configs gain max excess aligned among gainers
4 75 2 +0.5850 0
5 364 21 +1.0000 0
6 2233 148 +1.5850 0
7 13155 929 +2.3219 0
extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4 n=6: |KsK| 288/288
|M| 12/12 gain 3.000/3.000 excess +1.5850 b n=7: |KsK| 2400/2400 |M| 20/20 gain
5.000/5.000 excess +2.3219 b n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500
excess +2.9069 b n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor n=5: complete binary family 15 readers
(= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}: True n=6: complete binary family 31
readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}: True n=7: complete binary
family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}: True n=5: 3-block
readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2 witness:
[[ (0,), (1,), (2, 3, 4)], [(0, 2), (3,), (1, 4)]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[ (0,), (1,), (2, 3, 4, 5)], [(0, 2), (3,), (4,), (1, 5)]]

all sections reproduced in 40s
7. """Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as
exhaustive enumerates its full domain. Runtime ~2 min on one core.
"""
8.
9. import itertools, time from math import factorial, log2 from itertools import permutations,
combinations from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)

```

```

def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)
    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(1*m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))

# ----- partitions and Young subgroups -----
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcoiset(s,gens):
    """KsK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f,rest=el[0],el[1:]
    for r in setparts(rest):

```

```

        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
        yield [(f,)]+r
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:],size-1):
        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))+t
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L

# ----- reader records -----
def record_R1(t,L,P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        s=[]; y=x
        for _ in range(T): s.append(L[y]); y=t[y]
        out.append(tuple(s))
    return tuple(sorted(out))

T0=time.time()
print("=*74)
print("SECTION 3.  R1 consistent set = the double coset KsK   (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P,k in [([(0,1,2),(3,4,5]),1),([(0,3),(1,4),(2,5]),1),([(0,1),(2,3),(4,5]),2),
            ([[(0,1,2),(3,4,5)],3),([(0,2,4),(1,3,5)],1)]:
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record_R1(t,L,P)==record_R1(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    cf=young(P)**2//young(meet(P,imgP(P,s)))
    print(f"   P={P} sigma={k}:  |consistent|={len(cons):4d}  |KsK|={len(I):4d}   "
          f"closed form={cf:4d}  match={len(cons)==len(I)==cf}")

print("\n"+"=*74)
print("SECTION 4.  the ladder  G <= R3 <= R2 <= R1   (class reps x all set partitions)")
print("   n  configs   |R2|<|R1|   |R3|<|R2|       R3==G       R1==G")
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    tot=s21=s32=eqG=r1G=0; mult=Counter()
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens)

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

pc=[dcoset(p, gens) for p in pw]
R2=set()
for t in R1:
    q=t; ok=True
    for i in range(T):
        if q not in pc[i]: ok=False; break
        q=comp(t,q)
    if ok: R2.add(t)
tgt=itinerary(s,L,2*n)
R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
G=set(comp(k,comp(s,inv(k))) for k in K)
assert G<=R3<=R2<=R1
tot+=1
s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
eqG+= R3==G; r1G+= R1==G
mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
print(f"    {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
if n==6:
    print(f"    |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"="*74)
print("SECTION 5.  R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[(log2(young(P)**2//young(meet(P,imgP(P,s))))), aligned(P,s)) for P in
uparts(range(N),bs)]
        rows.sort(); K=log2(young([tuple(range(bs))]*(N//bs)))
        al=[r[0] for r in rows if r[1]]
        ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
        print(f"    sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
              f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"="*74)
print("SECTION 6.  passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b) "
          f"consistent {h:>9} ({log2(h):6.3f} b)  ratio {a/h:.1f}x")
print(f"    transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7.  joint reader census S_4..S_7, and the extremal family")
print("    n configs  gain  max excess  aligned among gainers")
for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:
            I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
            e=log2(len(I))-log2(M)-d; tot+=1
            if e>1e-9:
                sup+=1
                if aligned(P,s): alsup+=1
            mx=max(mx,e)
    print(f"    {n} {tot:8d} {sup:6d} {mx:+.4f} {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest,  gain=(n-2)(n-3)/4")
for n in (6,7,8,9):

```

```

    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"      n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
          f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

print("\n"+"="*74)
print("SECTION 8. families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"      n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"n-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
    three=[P for P in setparts(range(n)) if len(P)==3]
    kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    cos=[dcoset(s,blockgens(P,n)) for P in props]
    best=None
    for m in (1,2):
        for cb in combinations(range(len(props)),m):
            I=set(allp)
            for i in cb: I&=cos[i]
            if len(I)==1: best=(m,[props[i] for i in cb]); break
    if best: break
    print(f"      n={n}: 3-block readers breaking the tie: {kills}/{len(three)}; "
          f"minimum proper partitions for exact recovery = {best[0]}")
    print(f"      witness: {best[1]}")

# ===== additional sections for the expanded paper =====
def contingency(P,s):
    m=len(P); idx={}
    for i,b in enumerate(P):
        for x in b: idx[x]=i
    N=[[0]*m for _ in range(m)]
    for i,b in enumerate(P):
        for x in b: N[i][idx[s[x]]]+=1
    return tuple(tuple(r) for r in N)
def prodfac(N):
    r=1
    for row in N:
        for v in row: r*=factorial(v)
    return r
def undirected(s): return frozenset(frozenset((x,s[x])) for x in range(len(s)))
def necklaces(k):
    return {min(tuple(w[i:]+w[:i]) for i in range(k)) for w in itertools.product((0,1),repeat=k)}
def bracelets(k):
    out=set()

```

```

    for w in itertools.product((0,1),repeat=k):
        r=tuple(reversed(w))
        out.add(min([tuple(w[i:]+w[:i]) for i in range(k)]+[tuple(r[i:]+r[:i]) for i in range(k)]))
    return out
def twoblocks(n):
    out=set()
    for kk in range(1,n):
        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]

print("\n"+"="*74)
print("SECTION 3 / 4A.  the record IS the contingency table; product formula")
ok=True; checked=0
for n in (4,5):
    for P in setparts(range(n)):
        base_=1
        for b in P: base_*=factorial(len(b))
        for s in permutations(range(n)):
            N=contingency(P,s)
            if base_*base_//prodfac(N) != len(dcoset(s,blockgens(P,n))): ok=False
            checked+=1
print(f"   |KsK| = (prod|B|!)^2 / prod N_ij!  in all {checked} pairs at n=4,5 : {ok}")
n=6; s=tuple((x+1)%6 for x in range(6))
for P,name in [([(0,1,2),(3,4,5)],"A"),([(0,2,4),(1,3,5)],"B")]:
    N=contingency(P,s); K=1
    for b in P: K*=factorial(len(b))
    print(f"   worked example reader {name}: N={N}   |K|={K}   |KsK|={K*K//prodfac(N)}")

print("\n"+"="*74)
print("SECTION 4.  inversion criterion:  sigma^-1 in KsK  <=>  N(P,sigma) symmetric")
bad=tot=sym2=tot2=0
for n in (4,5,6):
    for P in setparts(range(n)):
        for s in permutations(range(n)):
            N=contingency(P,s); m=len(N)
            symm=all(N[i][j]==N[j][i] for i in range(m) for j in range(m))
            if symm != (inv(s) in dcoset(s,blockgens(P,n))): bad+=1
            tot+=1
            if m<=2:
                tot2+=1; sym2+= symm
print(f"   {tot} triples at n=4,5,6 : counterexamples = {bad}")
print(f"   of {tot2} triples with |P|<=2, table symmetric in {sym2} (100% expected)")

print("\n"+"="*74)
print("SECTION 9.  complete binary family recovers the undirected graph; |cap| = 2^c")
for n in (4,5,6,7):
    allp=list(permutations(range(n))); tb=twoblocks(n); allok=True
    for lam in parts_of(n):
        s=rep_of(lam,n); cap=set(allp)
        for P in tb: cap&=dcset(s,blockgens(P,n))
        c=sum(1 for L in lam if L>=3)
        allok &= (len(cap)==2**c) and (cap=={t for t in allp if undirected(t)==undirected(s)})
    print(f"   n={n} ({len(tb)} binary readers, {len(parts_of(n))} cycle types): |cap| = 2^c and cap
= orientations : {allok}")

print("\n"+"="*74)
print("SECTION 10.  handedness threshold")
for n in (5,6,7):
    s=tuple((x+1)%n for x in range(n)); T=order(s); allsym=True
    for P in twoblocks(n):
        q=s

```

```

    for t in range(T):
        N=contingency(P,q)
        if N[0][1]!=N[1][0]: allsym=False
        q=comp(s,q)
    print(f"    n={n}: N(P,sigma^t) symmetric for every power and every binary reader: {allsym}")
print("    k necklaces bracelets chiral?    R3 binary family on a k-cycle")
for kk in range(3,8):
    n=kk; allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n):
        L=[0]*n
        for i,b in enumerate(P):
            for x in b: L[x]=i
        tgt=itinerary(s,L,2*n)
        cap&={t for t in allp if itinerary(t,L,2*n)==tgt}
    Nn=len(necklaces(kk)); Bb=len(bracelets(kk))
    print(f"    {kk}    {Nn:8d}    {Bb:9d}    {str(Nn>Bb):7s}    |cap|={len(cap)}    exact={cap=={s}}")

print("\n"+"="*74)
print("SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy")
for n in (5,6):
    allp=list(permutations(range(n)))
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    mins=set(); dens=[]
    for lam in parts_of(n):
        s=rep_of(lam,n)
        cos=[dcoset(s,blockgens(P,n)) for P in props]
        singles=sum(1 for c in cos if len(c)==1)
        good=tot=0
        for i in range(len(props)):
            for j in range(i+1,len(props)):
                tot+=1; good+= len(cos[i]&cos[j])==1
            mins.add((singles==0, good>0)); dens.append((lam,good,tot))
    print(f"    n={n}: single reader never suffices and some pair always does:
{mins=={(True,True)}}")
    print(f"    pair success rate by cycle type: " + ", ".join(f"{l}:{100*g/t:.1f}%" for l,g,t
in dens[:4]) + " ...")
    for m in (2,3,4):
        fam=[P for P in setparts(range(n)) if len(P)==m]
        exact=all(len({t for t in allp} & set.intersection(*[dcoset(rep_of(lam,n),blockgens(P,n))
for P in fam]))==1
        for lam in parts_of(n):
            print(f"    complete {m}-block family ({len(fam):3d} readers): exact for every cycle
type = {exact}")
print(f"\nall sections reproduced in {time.time()-T0:.0f}s")
10. =====
SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma=+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma=+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma=+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma=+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma=+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True

=====
SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)
n configs |R2|<|R1| |R3|<|R2| R3==G R1==G 4 75 40 3
39 (52.0%) 5 5 364 262 18 201 (55.2%) 7 6 2233
1838 132 1205 (54.0%) 11 |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41,
4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72, 16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1} (no
non-integer ratios: True)

=====
SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)

```

```

sigma=+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
<+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
<+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio 6.0x
<+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 ( 7.755 b) ratio
16.0x transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
n configs gain max excess aligned among gainers
4 75 2 +0.5850 0
5 364 21 +1.0000 0
6 2233 148 +1.5850 0
7 13155 929 +2.3219 0
extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4 n=6: |KsK| 288/288
|M| 12/12 gain 3.000/3.000 excess +1.5850 b n=7: |KsK| 2400/2400 |M| 20/20 gain
5.000/5.000 excess +2.3219 b n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500
excess +2.9069 b n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor n=5: complete binary family 15 readers
(= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}: True n=6: complete binary family 31
readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}: True n=7: complete binary
family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}: True n=5: 3-block
readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2 witness:
[[ (0, ), (1, ), (2, 3, 4) ], [ (0, 2), (3, ), (1, 4) ]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[ (0, ), (1, ), (2, 3, 4, 5) ], [ (0, 2), (3, ), (4, ), (1, 5) ]]

=====
SECTION 3 / 4A. the record IS the contingency table; product formula |KsK| = (prod |B|!)^2 / prod
N_ij! in all 6600 pairs at n=4,5 : True worked example reader A: N=((2, 1), (1, 2)) |K|=36
|KsK|=324 worked example reader B: N=((0, 3), (3, 0)) |K|=36 |KsK|=36

=====
SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric 152760 triples at
n=4,5,6 : counterexamples = 0 of 25152 triples with |P|<=2, table symmetric in 25152 (100%
expected)

=====
SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c n=4 (7 binary
readers, 5 cycle types): |cap| = 2^c and cap = orientations : True n=5 (15 binary readers, 7
cycle types): |cap| = 2^c and cap = orientations : True n=6 (31 binary readers, 11 cycle types):
|cap| = 2^c and cap = orientations : True n=7 (63 binary readers, 15 cycle types): |cap| = 2^c
and cap = orientations : True

=====
SECTION 10. handedness threshold n=5: N(P,sigma^t) symmetric for every power and every binary
reader: True n=6: N(P,sigma^t) symmetric for every power and every binary reader: True n=7:
N(P,sigma^t) symmetric for every power and every binary reader: True k necklaces bracelets

```

```

chiral?   R3 binary family on a k-cycle   3           4           4   False   |cap|=2   exact=False
4         6           6   False   |cap|=2   exact=False   5           8           8   False   |cap|=2
exact=False   6           14           13   True   |cap|=1   exact=True   7           20           18
True   |cap|=1   exact=True

=====
SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy   n=5: single
reader never suffices and some pair always does: True   pair success rate by cycle type:
(5,):14.7%, (4, 1):14.7%, (3, 2):17.1%, (3, 1, 1):15.9% ...
    complete 2-block family ( 15 readers): exact for every cycle type = False   complete
3-block family ( 25 readers): exact for every cycle type = True   complete 4-block family ( 10
readers): exact for every cycle type = True   n=6: single reader never suffices and some pair
always does: True   pair success rate by cycle type: (6,):11.9%, (5, 1):11.0%, (4, 2):13.1%,
(4, 1, 1):12.2% ...
    complete 2-block family ( 31 readers): exact for every cycle type = False   complete
3-block family ( 90 readers): exact for every cycle type = True   complete 4-block family ( 65
readers): exact for every cycle type = True

all sections reproduced in 126s
11. """Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as
exhaustive enumerates its full domain. Runtime ~2 min on one core.
"""
12.
13. import itertools, time from math import factorial, log2 from itertools import permutations,
combinations from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)
    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(l**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))

# ----- partitions and Young subgroups -----

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcoset(s,gens):
    """KsK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f,rest=el[0],el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
    yield [(f,)]+r
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:],size-1):
        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))]+t
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L

# ----- reader records -----
def record_R1(t,L,P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t,L,T):

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

n=len(t); out=[]
for x in range(n):
    s=[]; y=x
    for _ in range(T): s.append(L[y]); y=t[y]
    out.append(tuple(s))
return tuple(sorted(out))

T0=time.time()
print("=*74)
print("SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P,k in [([(0,1,2),(3,4,5]),1),([(0,3),(1,4),(2,5]),1),([(0,1),(2,3),(4,5]),2),
            ([[(0,1,2),(3,4,5]),3),([(0,2,4),(1,3,5]),1]):
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record_R1(t,L,P)==record_R1(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    cf=young(P)**2//young(meet(P,imgP(P,s)))
    print(f" P={P} sigma={k}: |consistent|={len(cons):4d} |KsK|={len(I):4d} "
          f"closed form={cf:4d} match={len(cons)==len(I)==cf}")

print("\n"+"=*74)
print("SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)")
print(" n configs |R2|<|R1| |R3|<|R2| R3==G R1==G")
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    tot=s21=s32=eqG=r1G=0; mult=Counter()
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens)
            pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(k,comp(s,inv(k))) for k in K)
            assert G<=R3<=R2<=R1
            tot+=1
            s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
            eqG+= R3==G; r1G+= R1==G
            mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
    print(f" {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
    if n==6:
        print(f" |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"=*74)
print("SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[(log2(young(P)**2//young(meet(P,imgP(P,s))))), aligned(P,s)) for P in
uparts(range(N),bs)]
        rows.sort(); K=log2(young([tuple(range(bs))*N//bs]))

```

```

    al=[r[0] for r in rows if r[1]]
    ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
    print(f"    sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
          f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"="*74)
print("SECTION 6.  passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b)  "
          f"consistent {h:>9} ({log2(h):6.3f} b)  ratio {a/h:.1f}x")
print(f"    transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7.  joint reader census S_4..S_7, and the extremal family")
print("    n configs  gain  max excess  aligned among gainers")
for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:
            I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
            e=log2(len(I))-log2(M)-d; tot+=1
            if e>1e-9:
                sup+=1
                if aligned(P,s): alsup+=1
            mx=max(mx,e)
    print(f"    {n} {tot:8d} {sup:6d}    {mx:+.4f}    {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest,  gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"    n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
          f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

print("\n"+"="*74)
print("SECTION 8.  families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"    n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
    three=[P for P in setparts(range(n)) if len(P)==3]
    kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
    props=[P for P in setparts(range(n)) if 1<len(P)<n]

```

```

cos=[dcoset(s,blockgens(P,n)) for P in props]
best=None
for m in (1,2):
    for cb in combinations(range(len(props)),m):
        I=set(allp)
        for i in cb: I&=cos[i]
        if len(I)==1: best=(m,[props[i] for i in cb]); break
    if best: break
print(f"    n={n}: 3-block readers breaking the tie: {kills}/{len(three)};  "
      f"minimum proper partitions for exact recovery = {best[0]}")
print(f"        witness: {best[1]}")

# ===== additional sections for the expanded paper =====
def contingency(P,s):
    m=len(P); idx={}
    for i,b in enumerate(P):
        for x in b: idx[x]=i
    N=[[0]*m for _ in range(m)]
    for i,b in enumerate(P):
        for x in b: N[i][idx[s[x]]]+=1
    return tuple(tuple(r) for r in N)
def prodfac(N):
    r=1
    for row in N:
        for v in row: r*=factorial(v)
    return r
def undirected(s): return frozenset(frozenset((x,s[x])) for x in range(len(s)))
def necklaces(k):
    return {min(tuple(w[i:]+w[:i]) for i in range(k)) for w in itertools.product((0,1),repeat=k)}
def bracelets(k):
    out=set()
    for w in itertools.product((0,1),repeat=k):
        r=tuple(reversed(w))
        out.add(min([tuple(w[i:]+w[:i]) for i in range(k)]+[tuple(r[i:]+r[:i]) for i in range(k)]))
    return out
def twoblocks(n):
    out=set()
    for kk in range(1,n):
        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]

print("\n"+"="*74)
print("SECTION 3 / 4A.  the record IS the contingency table; product formula")
ok=True; checked=0
for n in (4,5):
    for P in setparts(range(n)):
        base_=1
        for b in P: base_*=factorial(len(b))
        for s in permutations(range(n)):
            N=contingency(P,s)
            if base_*base_//prodfac(N) != len(dcoset(s,blockgens(P,n))): ok=False
            checked+=1
print(f"    |KsK| = (prod|B|!)^2 / prod N_ij!  in all {checked} pairs at n=4,5 : {ok}")
n=6; s=tuple((x+1)%6 for x in range(6))
for P,name in [((0,1,2),(3,4,5)),"A"],((0,2,4),(1,3,5)),"B"]]:
    N=contingency(P,s); K=1
    for b in P: K*=factorial(len(b))
    print(f"    worked example reader {name}: N={N}  |K|={K}  |KsK|={K*K//prodfac(N)}")

```

```

print("\n"+"="*74)
print("SECTION 4. inversion criterion:  $\sigma^{-1}$  in  $KsK \iff N(P,\sigma)$  symmetric")
bad=tot=sym2=tot2=0
for n in (4,5,6):
    for P in setparts(range(n)):
        for s in permutations(range(n)):
            N=contingency(P,s); m=len(N)
            symm=all(N[i][j]==N[j][i] for i in range(m) for j in range(m))
            if symm != (inv(s) in dcoset(s,blockgens(P,n))): bad+=1
            tot+=1
            if m<=2:
                tot2+=1; sym2+= symm
print(f" {tot} triples at n=4,5,6 : counterexamples = {bad}")
print(f" of {tot2} triples with  $|P| \leq 2$ , table symmetric in {sym2} (100% expected)")

print("\n"+"="*74)
print("SECTION 9. complete binary family recovers the undirected graph;  $|cap| = 2^c$ ")
for n in (4,5,6,7):
    allp=list(permutations(range(n))); tb=twoblocks(n); allok=True
    for lam in parts_of(n):
        s=rep_of(lam,n); cap=set(allp)
        for P in tb: cap&=dcoset(s,blockgens(P,n))
        c=sum(1 for L in lam if L>=3)
        allok &= (len(cap)==2*c) and (cap=={t for t in allp if undirected(t)==undirected(s)})
    print(f" n={n} ({len(tb)} binary readers, {len(parts_of(n))} cycle types):  $|cap| = 2^c$  and cap
= orientations : {allok}")

print("\n"+"="*74)
print("SECTION 10. handedness threshold")
for n in (5,6,7):
    s=tuple((x+1)%n for x in range(n)); T=order(s); allsym=True
    for P in twoblocks(n):
        q=s
        for t in range(T):
            N=contingency(P,q)
            if N[0][1]!=N[1][0]: allsym=False
            q=comp(s,q)
    print(f" n={n}:  $N(P,\sigma^t)$  symmetric for every power and every binary reader: {allsym}")
print(" k necklaces bracelets chiral? R3 binary family on a k-cycle")
for kk in range(3,8):
    n=kk; allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n):
        L=[0]*n
        for i,b in enumerate(P):
            for x in b: L[x]=i
        tgt=itinerary(s,L,2*n)
        cap&={t for t in allp if itinerary(t,L,2*n)==tgt}
    Nn=len(necklaces(kk)); Bb=len(bracelets(kk))
    print(f" {kk} {Nn:8d} {Bb:9d} {str(Nn>Bb):7s}  $|cap|={len(cap)}$  exact={cap=={s}}")

print("\n"+"="*74)
print("SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy")
for n in (5,6):
    allp=list(permutations(range(n)))
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    mins=set(); dens=[]
    for lam in parts_of(n):
        s=rep_of(lam,n)
        cos=[dcoset(s,blockgens(P,n)) for P in props]
        singles=sum(1 for c in cos if len(c)==1)
        good=tot=0
        for i in range(len(props)):

```

```

        for j in range(i+1,len(props)):
            tot+=1; good+= len(cos[i]&cos[j])==1
            mins.add((singles==0, good>0)); dens.append((lam,good,tot))
        print(f"    n={n}: single reader never suffices and some pair always does:
{mins=={(True,True)}}")
        print(f"        pair success rate by cycle type: " + ", ".join(f"{l}:{100*g/t:.1f}%" for l,g,t
in dens[:4]) + " ...")
        for m in (2,3,4):
            fam=[P for P in setparts(range(n)) if len(P)==m]
            exact=all(len({t for t in allp} & set.intersection(*[dcoset(rep_of(lam,n),blockgens(P,n))
for P in fam]))==1
            for lam in parts_of(n)
            print(f"        complete {m}-block family ({len(fam):3d} readers): exact for every cycle
type = {exact}")
        print(f"\nall sections reproduced in {time.time()-T0:.0f}s")

print("\n"+"="*74)
print("PROPOSITION 5.1.  dichotomy behind |G| divides |R3|")
print("    n  configs  Case A   Case B   |R3|=|K| in B   divisibility failures")
for n in (4,5,6):
    allP=list(setparts(range(n))); A=B=Bok=viol=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                    q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            rem=set(R3); sizes=[]
            while rem:
                t=next(iter(rem))
                cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
                sizes.append(len(cl)); rem-=cl
            if len(set(sizes))==1 and sizes[0]==len(G): A+=1
            else:
                B+=1
                if len(R3)==len(K): Bok+=1
                if len(R3)%len(G)!=0: viol+=1
        print(f"    {n} {A+B:8d} {A:8d} {B:8d} {Bok:13d} {viol:20d}")

print("\n"+"="*74)
print("THEOREM 5.1.  |R3| = |K| for every sigma and P;  Corollaries 5.2, 5.3;  Prop 5.4")
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def alignedP(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
for n in (4,5,6):
    allP=list(setparts(range(n)))
    tot=idok=termok=c2=c4=triv=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)

```

```

R1=dcoset(s, gens); pc=[dcoset(p, gens) for p in pw]
R2=set()
for t in R1:
    q=t; ok=True
    for i in range(T):
        if q not in pc[i]: ok=False; break
        q=comp(t, q)
    if ok: R2.add(t)
H=2*n; tgt=itinerary(s, L, H)
R3=set(t for t in R2 if itinerary(t, L, H)==tgt)
G=set(comp(kk, comp(s, inv(kk))) for kk in K)
CK=set(kk for kk in K if comp(kk, s)==comp(s, kk))
words=Counter()
for x in range(n):
    w=[]; y=x
    for _ in range(H): w.append(L[y]); y=s[y]
    words[tuple(w)]+=1
assign=1
for i, b in enumerate(P):
    den=1
    for w, m in words.items():
        if w[0]==i: den*=factorial(m)
    assign*= factorial(len(b))/den
bij=1
for w, m in words.items(): bij*=factorial(m)
tot+=1
idok += len(R3)==len(K)
termok += assign*bij==len(K)==len(R3)
c2 += (R3==G)==(len(CK)==1)
a=alignedP(P, s)
c4 += (a==(len(R1)==len(K))==len(R2)==len(R3))
triv += len(CK)==1
print(f"    n={n}: {tot:5d} configs   |R3|=|K|: {idok}   proof terms match: {termok}"
      f"    Cor 5.3 holds: {c2}   Prop 5.4 holds: {c4}   C_K(sigma) trivial: {triv}"
      f"({100*triv/tot:.1f}%)")
14. =====
SECTION 3.  R1 consistent set = the double coset KsK (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma=+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma=+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma=+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma=+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma=+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True

=====
SECTION 4.  the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)
n   configs   |R2|<|R1|   |R3|<|R2|   R3==G   R1==G   4       75       40       3
39 (52.0%)    5       5       364       262       18       201 (55.2%)   7       6       2233
1838          132      1205 (54.0%)    11          |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41,
4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72, 16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1} (no
non-integer ratios: True)

=====
SECTION 5.  R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)
sigma=+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True

```

```

sigma=+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
  <+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
  <+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio 6.0x
  <+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 (7.755 b) ratio
16.0x transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
  n configs gain max excess aligned among gainers
  4 75 2 +0.5850 0
  5 364 21 +1.0000 0
  6 2233 148 +1.5850 0
  7 13155 929 +2.3219 0
  extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4 n=6: |KsK| 288/288
|M| 12/12 gain 3.000/3.000 excess +1.5850 b n=7: |KsK| 2400/2400 |M| 20/20 gain
5.000/5.000 excess +2.3219 b n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500
excess +2.9069 b n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor n=5: complete binary family 15 readers
(= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}: True n=6: complete binary family 31
readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}: True n=7: complete binary
family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}: True n=5: 3-block
readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2 witness:
[[ (0,), (1,), (2, 3, 4)], [(0, 2), (3,), (1, 4)]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[ (0,), (1,), (2, 3, 4, 5)], [(0, 2), (3,), (4,), (1, 5)]]

=====
SECTION 3 / 4A. the record IS the contingency table; product formula |KsK| = (prod|B|!)^2 / prod
N_ij! in all 6600 pairs at n=4,5 : True worked example reader A: N=((2, 1), (1, 2)) |K|=36
|KsK|=324 worked example reader B: N=((0, 3), (3, 0)) |K|=36 |KsK|=36

=====
SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric 152760 triples at
n=4,5,6 : counterexamples = 0 of 25152 triples with |P|<=2, table symmetric in 25152 (100%
expected)

=====
SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c n=4 (7 binary
readers, 5 cycle types): |cap| = 2^c and cap = orientations : True n=5 (15 binary readers, 7
cycle types): |cap| = 2^c and cap = orientations : True n=6 (31 binary readers, 11 cycle types):
|cap| = 2^c and cap = orientations : True n=7 (63 binary readers, 15 cycle types): |cap| = 2^c
and cap = orientations : True

=====
SECTION 10. handedness threshold n=5: N(P,sigma^t) symmetric for every power and every binary
reader: True n=6: N(P,sigma^t) symmetric for every power and every binary reader: True n=7:
N(P,sigma^t) symmetric for every power and every binary reader: True k necklaces bracelets
chiral? R3 binary family on a k-cycle 3 4 4 False |cap|=2 exact=False
4 6 6 False |cap|=2 exact=False 5 8 8 False |cap|=2
exact=False 6 14 13 True |cap|=1 exact=True 7 20 18
True |cap|=1 exact=True

=====
SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy n=5: single
reader never suffices and some pair always does: True pair success rate by cycle type:
(5,):14.7%, (4, 1):14.7%, (3, 2):17.1%, (3, 1, 1):15.9% ...
complete 2-block family ( 15 readers): exact for every cycle type = False complete

```

```

3-block family ( 25 readers): exact for every cycle type = True      complete 4-block family ( 10
readers): exact for every cycle type = True      n=6: single reader never suffices and some pair
always does: True      pair success rate by cycle type: (6,):11.9%, (5, 1):11.0%, (4, 2):13.1%,
(4, 1, 1):12.2% ...
      complete 2-block family ( 31 readers): exact for every cycle type = False      complete
3-block family ( 90 readers): exact for every cycle type = True      complete 4-block family ( 65
readers): exact for every cycle type = True

all sections reproduced in 125s

=====
PROPOSITION 5.1. dichotomy behind |G| divides |R3|
n   configs   Case A   Case B   |R3|=|K| in B   divisibility failures   4       75       63
12      12      0       5       364       301       63
0       6       2233    1869    364       364       0
=====
THEOREM 5.1. |R3| = |K| for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4   n=4:   75
configs |R3|=|K|: 75   proof terms match: 75   Cor 5.3 holds: 75   Prop 5.4 holds: 75   C_K(sigma)
trivial: 39 (52.0%)
n=5:   364 configs |R3|=|K|: 364   proof terms match: 364   Cor 5.3 holds: 364   Prop 5.4
holds: 364   C_K(sigma) trivial: 201 (55.2%)
n=6:   2233 configs |R3|=|K|: 2233   proof terms match: 2233   Cor 5.3 holds: 2233   Prop 5.4
holds: 2233   C_K(sigma) trivial: 1205 (54.0%)
15. """Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as
exhaustive enumerates its full domain. Runtime ~2 min on one core.
"""
16.
17. import itertools, time from math import factorial, log2 from itertools import permutations,
combinations from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)
    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(1**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return

```

```

        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
        return list(go(n,n))

# ----- partitions and Young subgroups -----
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcoset(s,gens):
    """KsK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f,rest=el[0],el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
        yield [(f,)]+r
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:],size-1):
        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))]+t
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L

```

```

# ----- reader records -----
def record_R1(t,L,P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        s=[]; y=x
        for _ in range(T): s.append(L[y]); y=t[y]
        out.append(tuple(s))
    return tuple(sorted(out))

T0=time.time()
print("=*74)
print("SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P,k in [([(0,1,2),(3,4,5)],1),([(0,3),(1,4),(2,5)],1),([(0,1),(2,3),(4,5)],2),
            ([[(0,1,2),(3,4,5)],3),([(0,2,4),(1,3,5)],1)]:
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record_R1(t,L,P)==record_R1(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    cf=young(P)**2//young(meet(P,imgP(P,s)))
    print(f" P={P} sigma={k}: |consistent|={len(cons):4d} |KsK|={len(I):4d} "
          f"closed form={cf:4d} match={len(cons)==len(I)==cf}")

print("\n"+"=*74)
print("SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)")
print(" n configs |R2|<|R1| |R3|<|R2| R3==G R1==G")
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    tot=s21=s32=eqG=r1G=0; mult=Counter()
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens)
            pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                    q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(k,comp(s,inv(k))) for k in K)
            assert G<=R3<=R2<=R1
            tot+=1
            s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
            eqG+= R3==G; r1G+= R1==G
            mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
    print(f" {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
    if n==6:
        print(f" |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"=*74)
print("SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):

```

```

s=[(x+k)%N for x in range(N)]
for bs in (6,4,3,2):
    rows=[(log2(young(P)**2//young(meet(P,imgP(P,s)))), aligned(P,s)) for P in
uparts(range(N),bs)]
    rows.sort(); K=log2(young([tuple(range(bs))]*(N//bs)))
    al=[r[0] for r in rows if r[1]]
    ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
    print(f"    sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
          f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"="*74)
print("SECTION 6.  passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b)    "
          f"consistent {h:>9} ({log2(h):6.3f} b)    ratio {a/h:.1f}x")
print(f"    transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7.  joint reader census S_4..S_7, and the extremal family")
print("    n configs    gain    max excess    aligned among gainers")
for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:
            I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
            e=log2(len(I))-log2(M)-d; tot+=1
            if e>1e-9:
                sup+=1
                if aligned(P,s): alsup+=1
            mx=max(mx,e)
        print(f"    {n} {tot:8d} {sup:6d}    {mx:+.4f}    {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"    n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
          f"    f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

print("\n"+"="*74)
print("SECTION 8.  families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"    n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"    f"-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))

```

```

cap=set(allp)
for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
three=[P for P in setparts(range(n)) if len(P)==3]
kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
props=[P for P in setparts(range(n)) if 1<len(P)<n]
cos=[dcoset(s,blockgens(P,n)) for P in props]
best=None
for m in (1,2):
    for cb in combinations(range(len(props)),m):
        I=set(allp)
        for i in cb: I&=cos[i]
        if len(I)==1: best=(m,[props[i] for i in cb]); break
    if best: break
print(f"    n={n}: 3-block readers breaking the tie: {kills}/{len(three)};  "
      f"minimum proper partitions for exact recovery = {best[0]}")
print(f"        witness: {best[1]}")

# ===== additional sections for the expanded paper =====
def contingency(P,s):
    m=len(P); idx={}
    for i,b in enumerate(P):
        for x in b: idx[x]=i
    N=[[0]*m for _ in range(m)]
    for i,b in enumerate(P):
        for x in b: N[i][idx[s[x]]]+=1
    return tuple(tuple(r) for r in N)
def prodfac(N):
    r=1
    for row in N:
        for v in row: r*=factorial(v)
    return r
def undirected(s): return frozenset(frozenset((x,s[x])) for x in range(len(s)))
def necklaces(k):
    return {min(tuple(w[i:]+w[:i]) for i in range(k)) for w in itertools.product((0,1),repeat=k)}
def bracelets(k):
    out=set()
    for w in itertools.product((0,1),repeat=k):
        r=tuple(reversed(w))
        out.add(min([tuple(w[i:]+w[:i]) for i in range(k)]+[tuple(r[i:]+r[:i]) for i in range(k)]))
    return out
def twoblocks(n):
    out=set()
    for kk in range(1,n):
        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]

print("\n"+"="*74)
print("SECTION 3 / 4A.  the record IS the contingency table; product formula")
ok=True; checked=0
for n in (4,5):
    for P in setparts(range(n)):
        base_=1
        for b in P: base_*=factorial(len(b))
        for s in permutations(range(n)):
            N=contingency(P,s)
            if base_*base_//prodfac(N) != len(dcoset(s,blockgens(P,n))): ok=False
            checked+=1
print(f"    |KsK| = (prod|B|!)^2 / prod N_ij!  in all {checked} pairs at n=4,5 : {ok}")
n=6; s=tuple((x+1)%6 for x in range(6))

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

for P,name in [((0,1,2),(3,4,5)),("A"),((0,2,4),(1,3,5)),("B")]:
    N=contingency(P,s); K=1
    for b in P: K*=factorial(len(b))
    print(f"    worked example reader {name}: N={N}   |K|={K}   |KsK|={K*K//prodfac(N)}")

print("\n"+"="*74)
print("SECTION 4. inversion criterion:  $\sigma^{-1}$  in KsK  $\Leftrightarrow$  N(P,sigma) symmetric")
bad=tot=sym2=tot2=0
for n in (4,5,6):
    for P in setparts(range(n)):
        for s in permutations(range(n)):
            N=contingency(P,s); m=len(N)
            symm=all(N[i][j]==N[j][i] for i in range(m) for j in range(m))
            if symm != (inv(s) in dcoset(s,blockgens(P,n))): bad+=1
            tot+=1
            if m<=2:
                tot2+=1; sym2+= symm
print(f"    {tot} triples at n=4,5,6 : counterexamples = {bad}")
print(f"    of {tot2} triples with |P|<=2, table symmetric in {sym2} (100% expected)")

print("\n"+"="*74)
print("SECTION 9. complete binary family recovers the undirected graph; |cap| =  $2^c$ ")
for n in (4,5,6,7):
    allp=list(permutations(range(n))); tb=twoblocks(n); allok=True
    for lam in parts_of(n):
        s=rep_of(lam,n); cap=set(allp)
        for P in tb: cap&=dcset(s,blockgens(P,n))
        c=sum(1 for L in lam if L>=3)
        allok &= (len(cap)==2**c) and (cap=={t for t in allp if undirected(t)==undirected(s)})
    print(f"    n={n} ({len(tb)} binary readers, {len(parts_of(n))} cycle types): |cap| =  $2^c$  and cap
= orientations : {allok}")

print("\n"+"="*74)
print("SECTION 10. handedness threshold")
for n in (5,6,7):
    s=tuple((x+1)%n for x in range(n)); T=order(s); allsym=True
    for P in twoblocks(n):
        q=s
        for t in range(T):
            N=contingency(P,q)
            if N[0][1]!=N[1][0]: allsym=False
            q=comp(s,q)
    print(f"    n={n}: N(P,sigma^t) symmetric for every power and every binary reader: {allsym}")
print("    k necklaces bracelets chiral? R3 binary family on a k-cycle")
for kk in range(3,8):
    n=kk; allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n):
        L=[0]*n
        for i,b in enumerate(P):
            for x in b: L[x]=i
        tgt=itinerary(s,L,2*n)
        cap&={t for t in allp if itinerary(t,L,2*n)==tgt}
    Nn=len(necklaces(kk)); Bb=len(bracelets(kk))
    print(f"    {kk}   {Nn:8d}   {Bb:9d}   {str(Nn>Bb):7s}   |cap|={len(cap)}   exact={cap=={s}}")

print("\n"+"="*74)
print("SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy")
for n in (5,6):
    allp=list(permutations(range(n)))
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    mins=set(); dens=[]
    for lam in parts_of(n):

```

```

s=rep_of(lam,n)
cos=[dcoset(s,blockgens(P,n)) for P in props]
singles=sum(1 for c in cos if len(c)==1)
good=tot=0
for i in range(len(props)):
    for j in range(i+1,len(props)):
        tot+=1; good+= len(cos[i]&cos[j])==1
    mins.add((singles==0, good>0)); dens.append((lam,good,tot))
print(f"    n={n}: single reader never suffices and some pair always does:
{mins=={(True,True)}}")
print(f"    pair success rate by cycle type: " + ", ".join(f"{l}:{100*g/t:.1f}%" for l,g,t
in dens[:4]) + " ...")
for m in (2,3,4):
    fam=[P for P in setparts(range(n)) if len(P)==m]
    exact=all(len({t for t in allp} & set.intersection(*[dcoset(rep_of(lam,n),blockgens(P,n))
for P in fam]))==1
    for lam in parts_of(n))
    print(f"    complete {m}-block family ({len(fam):3d} readers): exact for every cycle
type = {exact}")
print(f"\nall sections reproduced in {time.time()-T0:.0f}s")

print("\n"+"="*74)
print("SUPERSEDED (kept as Problem-6 data): equal vs unequal K-class sizes inside R3")
print("    n    configs    Case A    Case B    |R3|=|K| in B    divisibility failures")
for n in (4,5,6):
    allP=list(setparts(range(n))); A=B=Bok=viol=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            rem=set(R3); sizes=[]
            while rem:
                t=next(iter(rem))
                cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
                sizes.append(len(cl)); rem-=cl
            if len(set(sizes))==1 and sizes[0]==len(G): A+=1
            else:
                B+=1
                if len(R3)==len(K): Bok+=1
                if len(R3)%len(G)!=0: viol+=1
    print(f"    {n} {A+B:8d} {A:8d} {B:8d} {Bok:13d} {viol:20d}")

print("\n"+"="*74)
print("THEOREM 5.1. |R3| = |K| for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4")
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def alignedP(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
for n in (4,5,6):
    allP=list(setparts(range(n)))
    tot=idok=termok=c2=c4=triv=0
    for lam in parts_of(n):

```

```

s=rep_of(lam,n); T=order(s)
pw=[]; q=s
for _ in range(T): pw.append(q); q=comp(s,q)
for P in allP:
    K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
    R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
    R2=set()
    for t in R1:
        q=t; ok=True
        for i in range(T):
            if q not in pc[i]: ok=False; break
            q=comp(t,q)
        if ok: R2.add(t)
    H=2*n; tgt=itinerary(s,L,H)
    R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
    G=set(comp(kk,comp(s,inv(kk))) for kk in K)
    CK=set(kk for kk in K if comp(kk,s)==comp(s,kk))
    words=Counter()
    for x in range(n):
        w=[]; y=x
        for _ in range(H): w.append(L[y]); y=s[y]
        words[tuple(w)]+=1
    assign=1
    for i,b in enumerate(P):
        den=1
        for w,m in words.items():
            if w[0]==i: den*=factorial(m)
        assign*= factorial(len(b))//den
    bij=1
    for w,m in words.items(): bij*=factorial(m)
    tot+=1
    idok += len(R3)==len(K)
    termok += assign*bij==len(K)==len(R3)
    c2 += (R3==G)==(len(CK)==1)
    a=alignedP(P,s)
    c4 += (a==(len(R1)==len(K))==(R2==R1)==(R3==R1))
    triv += len(CK)==1
    print(f"    n={n}: {tot:5d} configs    |R3|=|K|: {idok}    proof terms match: {termok}"
          f"    Cor 5.3 holds: {c2}    Prop 5.4 holds: {c4}    C_K(sigma) trivial: {triv}"
          f"({100*triv/tot:.1f}%)")

print("\n"+"="*74)
print("THEOREM 5.5 / COROLLARY 5.6.  structure of R3 from the word-class shift orbits")
def conj_classes_of_Sm(m):
    out=[]
    def pr(n,mx):
        if n==0: yield (); return
        for kk in range(min(n,mx),0,-1):
            for t in pr(n-kk,kk): yield (kk,)+t
    for lam in pr(m,m):
        c=Counter(lam); d=1
        for l,mu in c.items(): d*=(l**mu)*factorial(mu)
        out.append(factorial(m)//d)
    return out
def pcount(m,mx=None):
    mx=m if mx is None else mx
    if m==0: return 1
    return sum(pcount(m-kk,kk) for kk in range(min(m,mx),0,-1))
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=sz=cnt=corr=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s

```

```

for _ in range(T): pw.append(q); q=comp(s,q)
for P in allP:
    K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
    R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
    R2=set()
    for t in R1:
        q=t; ok=True
        for i in range(T):
            if q not in pc[i]: ok=False; break
            q=comp(t,q)
        if ok: R2.add(t)
    H=2*n; tgt=itinerary(s,L,H)
    R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
    rem=set(R3); obs=[]
    while rem:
        t=next(iter(rem))
        cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
        obs.append(len(cl)); rem-=cl
    word={}
    for x in range(n):
        w=[]; y=x
        for _ in range(H): w.append(L[y]); y=s[y]
        word[x]=tuple(w)
    cls={}
    for x in range(n): cls.setdefault(word[x],[]).append(x)
    nxt={w: word[s[cls[w][0]]] for w in cls}
    seen=set(); ms=[]
    for w in cls:
        if w in seen: continue
        y=w
        while y not in seen: seen.add(y); y=nxt[y]
        ms.append(len(cls[w]))
    pred=[]
    for combo in itertools.product(*[conj_classes_of_Sm(m) for m in ms]):
        cent=1
        for cs,m in zip(combo,ms): cent*= factorial(m)//cs
        pred.append(len(K)//cent)
    tot+=1
    sz += sorted(obs)==sorted(pred)
    npred=1
    for m in ms: npred*=pcount(m)
    cnt += len(obs)==npred
    corr += (len(set(obs))==1)==(max(ms)<=2)
    print(f"  n={n}: {tot:5d} configs  class-size multiset predicted: {sz}  class count = prod
p(m): {cnt}"
        f"  equal sizes iff all word-classes <= 2: {corr}")
18. =====
SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma=+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma=+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma=+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma=+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma=+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
=====
SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)
n configs |R2|<|R1| |R3|<|R2| R3==G R1==G 4 75 40 3
39 (52.0%) 5 5 364 262 18 201 (55.2%) 7 6 2233
1838 132 1205 (54.0%) 11 |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41,
4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72, 16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1} (no
non-integer ratios: True)
=====

```

```

SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)
  sigma=+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
  sigma=+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
  sigma=+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
  sigma=+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
  sigma=+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
  sigma=+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
  sigma=+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
  sigma=+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
  sigma=+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
  sigma=+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
  sigma=+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
  sigma=+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
  <+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
  <+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio 6.0x
  <+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 ( 7.755 b) ratio
16.0x transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
  n   configs   gain   max excess   aligned among gainers
  4     75       2     +0.5850       0
  5    364      21     +1.0000       0
  6   2233     148     +1.5850       0
  7  13155     929     +2.3219       0
  extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4      n=6: |KsK| 288/288
|M| 12/12 gain 3.000/3.000 excess +1.5850 b      n=7: |KsK| 2400/2400 |M| 20/20 gain
5.000/5.000 excess +2.3219 b      n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500
excess +2.9069 b      n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor      n=5: complete binary family 15 readers
(= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}: True      n=6: complete binary family 31
readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}: True      n=7: complete binary
family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}: True      n=5: 3-block
readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2      witness:
[[ (0,), (1,), (2, 3, 4)], [(0, 2), (3,), (1, 4)]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[ (0,), (1,), (2, 3, 4, 5)], [(0, 2), (3,), (4,), (1, 5)]]

=====
SECTION 3 / 4A. the record IS the contingency table; product formula      |KsK| = (prod|B|!)^2 / prod
N_ij! in all 6600 pairs at n=4,5 : True      worked example reader A: N=((2, 1), (1, 2))      |K|=36
|KsK|=324      worked example reader B: N=((0, 3), (3, 0))      |K|=36      |KsK|=36

=====
SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric      152760 triples at
n=4,5,6 : counterexamples = 0      of 25152 triples with |P|<=2, table symmetric in 25152 (100%
expected)

=====
SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c      n=4 (7 binary
readers, 5 cycle types): |cap| = 2^c and cap = orientations : True      n=5 (15 binary readers, 7
cycle types): |cap| = 2^c and cap = orientations : True      n=6 (31 binary readers, 11 cycle types):
|cap| = 2^c and cap = orientations : True      n=7 (63 binary readers, 15 cycle types): |cap| = 2^c
and cap = orientations : True

=====
SECTION 10. handedness threshold      n=5: N(P,sigma^t) symmetric for every power and every binary
reader: True      n=6: N(P,sigma^t) symmetric for every power and every binary reader: True      n=7:

```

```

N(P,sigma^t) symmetric for every power and every binary reader: True k necklaces bracelets
chiral? R3 binary family on a k-cycle 3 4 4 False |cap|=2 exact=False
4 6 6 False |cap|=2 exact=False 5 8 8 False |cap|=2
exact=False 6 14 13 True |cap|=1 exact=True 7 20 18
True |cap|=1 exact=True

=====
SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy n=5: single
reader never suffices and some pair always does: True pair success rate by cycle type:
(5,):14.7%, (4, 1):14.7%, (3, 2):17.1%, (3, 1, 1):15.9% ...
complete 2-block family ( 15 readers): exact for every cycle type = False complete
3-block family ( 25 readers): exact for every cycle type = True complete 4-block family ( 10
readers): exact for every cycle type = True n=6: single reader never suffices and some pair
always does: True pair success rate by cycle type: (6,):11.9%, (5, 1):11.0%, (4, 2):13.1%,
(4, 1, 1):12.2% ...
complete 2-block family ( 31 readers): exact for every cycle type = False complete
3-block family ( 90 readers): exact for every cycle type = True complete 4-block family ( 65
readers): exact for every cycle type = True

all sections reproduced in 126s

=====
SUPERSEDED (kept as Problem-6 data): equal vs unequal K-class sizes inside R3 n configs Case
A Case B |R3|=|K| in B divisibility failures 4 75 63 12 12
0 5 364 301 63 63 0 6 2233 1869 364
364 0

=====
THEOREM 5.1. |R3| = |K| for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4 n=4: 75
configs |R3|=|K|: 75 proof terms match: 75 Cor 5.3 holds: 75 Prop 5.4 holds: 75 C_K(sigma)
trivial: 39 (52.0%)
n=5: 364 configs |R3|=|K|: 364 proof terms match: 364 Cor 5.3 holds: 364 Prop 5.4
holds: 364 C_K(sigma) trivial: 201 (55.2%)
n=6: 2233 configs |R3|=|K|: 2233 proof terms match: 2233 Cor 5.3 holds: 2233 Prop 5.4
holds: 2233 C_K(sigma) trivial: 1205 (54.0%)

=====
THEOREM 5.5 / COROLLARY 5.6. structure of R3 from the word-class shift orbits n=4: 75 configs
class-size multiset predicted: 75 class count = prod p(m): 75 equal sizes iff all word-classes
<= 2: 75 n=5: 364 configs class-size multiset predicted: 364 class count = prod p(m): 364
equal sizes iff all word-classes <= 2: 364 n=6: 2233 configs class-size multiset predicted:
2233 class count = prod p(m): 2233 equal sizes iff all word-classes <= 2: 2233
19. """Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as
exhaustive enumerates its full domain. Runtime ~2 min on one core.
"""
20.
21. import itertools, time from math import factorial, log2 from itertools import permutations,
combinations from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)

```

```

    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(1**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))

# ----- partitions and Young subgroups -----
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcoset(s,gens):
    """KsK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f,rest=el[0],el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
        yield [(f,)]+r
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:],size-1):

```

```

        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))]+t
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L

# ----- reader records -----
def record_R1(t,L,P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        s=[]; y=x
        for _ in range(T): s.append(L[y]); y=t[y]
        out.append(tuple(s))
    return tuple(sorted(out))

T0=time.time()
print("=*74)
print("SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P,k in [([(0,1,2),(3,4,5]),1),([(0,3),(1,4),(2,5]),1),([(0,1),(2,3),(4,5]),2),
            ([[(0,1,2),(3,4,5)],3),([(0,2,4),(1,3,5)],1)]:
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record_R1(t,L,P)==record_R1(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    cf=young(P)**2//young(meet(P,imgP(P,s)))
    print(f" P={P} sigma={k}: |consistent|={len(cons):4d} |KsK|={len(I):4d} "
          f"closed form={cf:4d} match={len(cons)==len(I)==cf}")

print("\n"+"=*74)
print("SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)")
print(" n configs |R2|<|R1| |R3|<|R2| R3==G R1==G")
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    tot=s21=s32=eqG=r1G=0; mult=Counter()
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens)
            pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

        if ok: R2.add(t)
        tgt=itinerary(s,L,2*n)
        R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
        G=set(comp(k,comp(s,inv(k))) for k in K)
        assert G<=R3<=R2<=R1
        tot+=1
        s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
        eqG+= R3==G; r1G+= R1==G
        mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
    print(f"    {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
    if n==6:
        print(f"        |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"="*74)
print("SECTION 5.  R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[(log2(young(P)**2//young(meet(P,imgP(P,s))))), aligned(P,s)) for P in
uparts(range(N),bs)]
        rows.sort(); K=log2(young([tuple(range(bs))]*(N//bs)))
        al=[r[0] for r in rows if r[1]]
        ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
        print(f"        sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
            f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"="*74)
print("SECTION 6.  passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b) "
        f"consistent {h:>9} ({log2(h):6.3f} b)  ratio {a/h:.1f}x")
print(f"    transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7.  joint reader census S_4..S_7, and the extremal family")
print("    n configs  gain  max excess  aligned among gainers")
for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:
            I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
            e=log2(len(I))-log2(M)-d; tot+=1
            if e>1e-9:
                sup+=1
                if aligned(P,s): alsup+=1
            mx=max(mx,e)
    print(f"    {n} {tot:8d} {sup:6d} {mx:+.4f} {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest,  gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"        n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
        f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

```

```

print("\n"+"="*74)
print("SECTION 8.  families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"    n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
    three=[P for P in setparts(range(n)) if len(P)==3]
    kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    cos=[dcoset(s,blockgens(P,n)) for P in props]
    best=None
    for m in (1,2):
        for cb in combinations(range(len(props)),m):
            I=set(allp)
            for i in cb: I&=cos[i]
            if len(I)==1: best=(m,[props[i] for i in cb]); break
        if best: break
    print(f"    n={n}: 3-block readers breaking the tie: {kills}/{len(three)}; "
          f"minimum proper partitions for exact recovery = {best[0]}")
    print(f"        witness: {best[1]}")

# ===== additional sections for the expanded paper =====
def contingency(P,s):
    m=len(P); idx={}
    for i,b in enumerate(P):
        for x in b: idx[x]=i
    N=[[0]*m for _ in range(m)]
    for i,b in enumerate(P):
        for x in b: N[i][idx[s[x]]]+=1
    return tuple(tuple(r) for r in N)
def prodfac(N):
    r=1
    for row in N:
        for v in row: r*=factorial(v)
    return r
def undirected(s): return frozenset(frozenset((x,s[x])) for x in range(len(s)))
def necklaces(k):
    return {min(tuple(w[i:]+w[:i]) for i in range(k)) for w in itertools.product((0,1),repeat=k)}
def bracelets(k):
    out=set()
    for w in itertools.product((0,1),repeat=k):
        r=tuple(reversed(w))
        out.add(min([tuple(w[i:]+w[:i]) for i in range(k)]+[tuple(r[i:]+r[:i]) for i in range(k)]))
    return out
def twoblocks(n):
    out=set()
    for kk in range(1,n):

```

```

        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
        return [sorted(map(tuple,f)) for f in out]

print("\n"+"="*74)
print("SECTION 3 / 4A.  the record IS the contingency table; product formula")
ok=True; checked=0
for n in (4,5):
    for P in setparts(range(n)):
        base_=1
        for b in P: base_*=factorial(len(b))
        for s in permutations(range(n)):
            N=contingency(P,s)
            if base_*base_//prodfac(N) != len(dcoset(s,blockgens(P,n))): ok=False
            checked+=1
print(f"    |KsK| = (prod|B|!)^2 / prod N_ij!  in all {checked} pairs at n=4,5 : {ok}")
n=6; s=tuple((x+1)%6 for x in range(6))
for P,name in [((0,1,2),(3,4,5)),"A"],((0,2,4),(1,3,5)),"B"]]:
    N=contingency(P,s); K=1
    for b in P: K*=factorial(len(b))
    print(f"    worked example reader {name}: N={N}   |K|={K}   |KsK|={K*K//prodfac(N)}")

print("\n"+"="*74)
print("SECTION 4.  inversion criterion:  sigma^-1 in KsK  <=>  N(P,sigma) symmetric")
bad=tot=sym2=tot2=0
for n in (4,5,6):
    for P in setparts(range(n)):
        for s in permutations(range(n)):
            N=contingency(P,s); m=len(N)
            symm=all(N[i][j]==N[j][i] for i in range(m) for j in range(m))
            if symm != (inv(s) in dcoset(s,blockgens(P,n))): bad+=1
            tot+=1
            if m<=2:
                tot2+=1; sym2+= symm
print(f"    {tot} triples at n=4,5,6 : counterexamples = {bad}")
print(f"    of {tot2} triples with |P|<=2, table symmetric in {sym2} (100% expected)")

print("\n"+"="*74)
print("SECTION 9.  complete binary family recovers the undirected graph; |cap| = 2^c")
for n in (4,5,6,7):
    allp=list(permutations(range(n))); tb=twoblocks(n); allok=True
    for lam in parts_of(n):
        s=rep_of(lam,n); cap=set(allp)
        for P in tb: cap&=dcset(s,blockgens(P,n))
        c=sum(1 for L in lam if L>=3)
        allok &= (len(cap)==2**c) and (cap=={t for t in allp if undirected(t)==undirected(s)})
    print(f"    n={n} ({len(tb)} binary readers, {len(parts_of(n))} cycle types): |cap| = 2^c and cap
= orientations : {allok}")

print("\n"+"="*74)
print("SECTION 10.  handedness threshold")
for n in (5,6,7):
    s=tuple((x+1)%n for x in range(n)); T=order(s); allsym=True
    for P in twoblocks(n):
        q=s
        for t in range(T):
            N=contingency(P,q)
            if N[0][1]!=N[1][0]: allsym=False
            q=comp(s,q)
    print(f"    n={n}: N(P,sigma^t) symmetric for every power and every binary reader: {allsym}")
print("    k necklaces bracelets chiral?  R3 binary family on a k-cycle")
for kk in range(3,8):

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

n=kk; allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
cap=set(allp)
for P in twoblocks(n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    tgt=itinerary(s,L,2*n)
    cap&={t for t in allp if itinerary(t,L,2*n)==tgt}
Nn=len(necklaces(kk)); Bb=len(bracelets(kk))
print(f"    {kk}    {Nn:8d}    {Bb:9d}    {str(Nn>Bb):7s}    |cap|={len(cap)}    exact={cap=={s}}")

print("\n"+"="*74)
print("SECTION 11 / 11A.  families: minimum size, pair density, and the m-block dichotomy")
for n in (5,6):
    allp=list(permutations(range(n)))
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    mins=set(); dens=[]
    for lam in parts_of(n):
        s=rep_of(lam,n)
        cos=[dcoset(s,blockgens(P,n)) for P in props]
        singles=sum(1 for c in cos if len(c)==1)
        good=tot=0
        for i in range(len(props)):
            for j in range(i+1,len(props)):
                tot+=1; good+= len(cos[i]&cos[j])==1
            mins.add((singles==0, good>0)); dens.append((lam,good,tot))
    print(f"    n={n}:  single reader never suffices and some pair always does:
{mins=={(True,True)}}")
    print(f"    pair success rate by cycle type: " + ", ".join(f"{l}:{100*g/t:.1f}%" for l,g,t
in dens[:4]) + " ...")
    for m in (2,3,4):
        fam=[P for P in setparts(range(n)) if len(P)==m]
        exact=all(len({t for t in allp} & set.intersection(*[dcoset(rep_of(lam,n),blockgens(P,n))
for P in fam]))==1
        for lam in parts_of(n))
        print(f"    complete {m}-block family ({len(fam):3d} readers):  exact for every cycle
type = {exact}")
    print(f"\nall sections reproduced in {time.time()-T0:.0f}s")

print("\n"+"="*74)
print("SUPERSEDED (kept as Problem-6 data):  equal vs unequal K-class sizes inside R3")
print("    n  configs  Case A   Case B   |R3|=|K| in B   divisibility failures")
for n in (4,5,6):
    allP=list(setparts(range(n))); A=B=Bok=viol=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                    q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            rem=set(R3); sizes=[]
            while rem:

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

        t=next(iter(rem))
        cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
        sizes.append(len(cl)); rem-=cl
    if len(set(sizes))==1 and sizes[0]==len(G): A+=1
    else:
        B+=1
        if len(R3)==len(K): Bok+=1
        if len(R3)%len(G)!=0: viol+=1
    print(f"    {n} {A+B:8d} {A:8d} {B:8d} {Bok:13d} {viol:20d}")

print("\n"+"="*74)
print("THEOREM 5.1. |R3| = |K| for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4")
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def alignedP(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
for n in (4,5,6):
    allP=list(setparts(range(n)))
    tot=idok=termok=c2=c4=triv=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            H=2*n; tgt=itinerary(s,L,H)
            R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            CK=set(kk for kk in K if comp(kk,s)==comp(s,kk))
            words=Counter()
            for x in range(n):
                w=[]; y=x
                for _ in range(H): w.append(L[y]); y=s[y]
                words[tuple(w)]+=1
            assign=1
            for i,b in enumerate(P):
                den=1
                for w,m in words.items():
                    if w[0]==i: den*=factorial(m)
                assign*= factorial(len(b))/den
            bij=1
            for w,m in words.items(): bij*=factorial(m)
            tot+=1
            idok += len(R3)==len(K)
            termok += assign*bij==len(K)==len(R3)
            c2 += (R3==G)==(len(CK)==1)
            a=alignedP(P,s)
            c4 += (a==(len(R1)==len(K))==(R2==R1)==(R3==R1))
            triv += len(CK)==1
    print(f"    n={n}: {tot:5d} configs |R3|=|K|: {idok} proof terms match: {termok}"
          f"    Cor 5.3 holds: {c2} Prop 5.4 holds: {c4} C_K(sigma) trivial: {triv}"
          f"({100*triv/tot:.1f}%)")

print("\n"+"="*74)
print("THEOREM 5.5 / COROLLARY 5.6. structure of R3 from the word-class shift orbits")
def conj_classes_of_Sm(m):
    out=[]

```

```

def pr(n,mx):
    if n==0: yield (); return
    for kk in range(min(n,mx),0,-1):
        for t in pr(n-kk,kk): yield (kk,)+t
    for lam in pr(m,m):
        c=Counter(lam); d=1
        for l,mu in c.items(): d*=(1**mu)*factorial(mu)
        out.append(factorial(m)//d)
    return out
def pcount(m,mx=None):
    mx=m if mx is None else mx
    if m==0: return 1
    return sum(pcount(m-kk,kk) for kk in range(min(m,mx),0,-1))
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=sz=cnt=corr=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            H=2*n; tgt=itinerary(s,L,H)
            R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
            rem=set(R3); obs=[]
            while rem:
                t=next(iter(rem))
                cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
                obs.append(len(cl)); rem-=cl
            word={}
            for x in range(n):
                w=[]; y=x
                for _ in range(H): w.append(L[y]); y=s[y]
                word[x]=tuple(w)
            cls={}
            for x in range(n): cls.setdefault(word[x],[]).append(x)
            nxt={w: word[s[cls[w][0]]] for w in cls}
            seen=set(); ms=[]
            for w in cls:
                if w in seen: continue
                y=w
                while y not in seen: seen.add(y); y=nxt[y]
                ms.append(len(cls[w]))
            pred=[]
            for combo in itertools.product(*[conj_classes_of_Sm(m) for m in ms]):
                cent=1
                for cs,m in zip(combo,ms): cent*= factorial(m)//cs
                pred.append(len(K)//cent)
            tot+=1
            sz += sorted(obs)==sorted(pred)
            npred=1
            for m in ms: npred*=pcount(m)
            cnt += len(obs)==npred
            corr += (len(set(obs))==1)==(max(ms)<=2)
    print(f"    n={n}: {tot:5d} configs    class-size multiset predicted: {sz}    class count = prod
p(m): {cnt}")

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

    f"    equal sizes iff all word-classes <= 2: {corr}")

print("\n"+"="*74)
print("PROOF STEPS FOR THEOREM 5.5 (holonomy argument)")
print("    S1: K transitive on assignments, stabiliser = prod of symmetric groups on word-classes")
print("    S2: C_K(tau) <= Stab(A_tau)    (tau's own word-classes, not sigma's)")
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=s1=s2=s2wrong=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            H=2*n; tgt=itinerary(s,L,H)
            R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
            def words_of(t):
                d={}
                for x in range(n):
                    w=[]; y=x
                    for _ in range(H): w.append(L[y]); y=t[y]
                    d[x]=tuple(w)
                return d
            ws=words_of(s); cls={}
            for x in range(n): cls.setdefault(ws[x],[]).append(x)
            fib=[kk for kk in K if all(ws[kk[x]]==ws[x] for x in range(n))]
            exp=1
            for w,v in cls.items(): exp*=factorial(len(v))
            assign=1
            for i,b in enumerate(P):
                den=1
                for w,v in cls.items():
                    if w[0]==i: den*=factorial(len(v))
                assign*=factorial(len(b))/den
            tot+=1
            if len(fib)==exp and len(K)//len(fib)==assign: s1+=1
            okc=okw=True
            for t in R3:
                wt=words_of(t)
                for kk in K:
                    if comp(kk,t)==comp(t,kk):
                        if any(wt[kk[x]]!=wt[x] for x in range(n)): okc=False
                        if any(ws[kk[x]]!=ws[x] for x in range(n)): okw=False
            s2+=okc; s2wrong+= not okw
    print(f"    n={n}: {tot:5d} configs    S1: {s1}    S2 (tau's classes): {s2}"
          f"    configs where the sigma-version of S2 fails: {s2wrong}")

22. =====
SECTION 3. R1 consistent set = the double coset KsK    (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma=+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma=+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma=+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma=+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma=+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
=====

```

```

SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)
n configs |R2|<|R1| |R3|<|R2| R3==G R1==G 4 75 40 3
39 (52.0%) 5 5 364 262 18 201 (55.2%) 7 6 2233
1838 132 1205 (54.0%) 11 |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41,
4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72, 16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1} (no
non-integer ratios: True)

=====
SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)
sigma=+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
<+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
<+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio 6.0x
<+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 ( 7.755 b) ratio
16.0x transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
n configs gain max excess aligned among gainers
4 75 2 +0.5850 0
5 364 21 +1.0000 0
6 2233 148 +1.5850 0
7 13155 929 +2.3219 0
extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4 n=6: |KsK| 288/288
|M| 12/12 gain 3.000/3.000 excess +1.5850 b n=7: |KsK| 2400/2400 |M| 20/20 gain
5.000/5.000 excess +2.3219 b n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500
excess +2.9069 b n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor n=5: complete binary family 15 readers
(= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}: True n=6: complete binary family 31
readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}: True n=7: complete binary
family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}: True n=5: 3-block
readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2 witness:
[[{0,}, {1,}, {2, 3, 4}], [{0, 2}, {3,}, {1, 4}]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[{0,}, {1,}, {2, 3, 4, 5}], [{0, 2}, {3,}, {4,}, {1, 5}]]

=====
SECTION 3 / 4A. the record IS the contingency table; product formula |KsK| = (prod|B|!)^2 / prod
N_ij! in all 6600 pairs at n=4,5 : True worked example reader A: N=((2, 1), (1, 2)) |K|=36
|KsK|=324 worked example reader B: N=((0, 3), (3, 0)) |K|=36 |KsK|=36

=====
SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric 152760 triples at
n=4,5,6 : counterexamples = 0 of 25152 triples with |P|<=2, table symmetric in 25152 (100%
expected)

=====
SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c n=4 (7 binary

```

```

readers, 5 cycle types): |cap| = 2^c and cap = orientations : True    n=5 (15 binary readers, 7
cycle types): |cap| = 2^c and cap = orientations : True    n=6 (31 binary readers, 11 cycle types):
|cap| = 2^c and cap = orientations : True    n=7 (63 binary readers, 15 cycle types): |cap| = 2^c
and cap = orientations : True

=====
SECTION 10. handedness threshold    n=5: N(P,sigma^t) symmetric for every power and every binary
reader: True    n=6: N(P,sigma^t) symmetric for every power and every binary reader: True    n=7:
N(P,sigma^t) symmetric for every power and every binary reader: True    k necklaces bracelets
chiral?    R3 binary family on a k-cycle    3    4    4    False    |cap|=2    exact=False
4    6    6    False    |cap|=2    exact=False    5    8    8    False    |cap|=2
exact=False    6    14    13    True    |cap|=1    exact=True    7    20    18
True    |cap|=1    exact=True

=====
SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy    n=5: single
reader never suffices and some pair always does: True    pair success rate by cycle type:
(5,):14.7%, (4, 1):14.7%, (3, 2):17.1%, (3, 1, 1):15.9% ...
    complete 2-block family ( 15 readers): exact for every cycle type = False    complete
3-block family ( 25 readers): exact for every cycle type = True    complete 4-block family ( 10
readers): exact for every cycle type = True    n=6: single reader never suffices and some pair
always does: True    pair success rate by cycle type: (6,):11.9%, (5, 1):11.0%, (4, 2):13.1%,
(4, 1, 1):12.2% ...
    complete 2-block family ( 31 readers): exact for every cycle type = False    complete
3-block family ( 90 readers): exact for every cycle type = True    complete 4-block family ( 65
readers): exact for every cycle type = True

all sections reproduced in 126s

=====
SUPERSEDED (kept as Problem-6 data): equal vs unequal K-class sizes inside R3    n    configs    Case
A    Case B    |R3|=|K| in B    divisibility failures    4    75    63    12    12
0    5    364    301    63    63    0    6    2233    1869    364
364    0

=====
THEOREM 5.1. |R3| = |K| for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4    n=4:    75
configs |R3|=|K|: 75    proof terms match: 75    Cor 5.3 holds: 75    Prop 5.4 holds: 75    C_K(sigma)
trivial: 39 (52.0%)
    n=5:    364 configs |R3|=|K|: 364    proof terms match: 364    Cor 5.3 holds: 364    Prop 5.4
holds: 364    C_K(sigma) trivial: 201 (55.2%)
    n=6:    2233 configs |R3|=|K|: 2233    proof terms match: 2233    Cor 5.3 holds: 2233    Prop 5.4
holds: 2233    C_K(sigma) trivial: 1205 (54.0%)

=====
THEOREM 5.5 / COROLLARY 5.6. structure of R3 from the word-class shift orbits    n=4:    75 configs
class-size multiset predicted: 75    class count = prod p(m): 75    equal sizes iff all word-classes
<= 2: 75    n=5:    364 configs class-size multiset predicted: 364    class count = prod p(m): 364
equal sizes iff all word-classes <= 2: 364    n=6:    2233 configs class-size multiset predicted:
2233    class count = prod p(m): 2233    equal sizes iff all word-classes <= 2: 2233

=====
PROOF STEPS FOR THEOREM 5.5 (holonomy argument)
    S1: K transitive on assignments, stabiliser = prod of symmetric groups on word-classes    S2:
C_K(tau) <= Stab(A_tau) (tau's own word-classes, not sigma's)
    n=4:    75 configs    S1: 75    S2 (tau's classes): 75    configs where the sigma-version of S2
fails: 6
    n=5:    364 configs    S1: 364    S2 (tau's classes): 364    configs where the sigma-version of S2
fails: 52    n=6:    2233 configs    S1: 2233    S2 (tau's classes): 2233    configs where the sigma-
version of S2 fails: 478
23. ""Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as

```

exhaustive enumerates its full domain. Runtime ~2 min on one core.

```

"""
24.
25. import itertools, time
from math import factorial, log2
from itertools import permutations, combinations
from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)
    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(1**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))

# ----- partitions and Young subgroups -----
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]
    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
        out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

def dcoiset(s, gens):
    """KsK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f, rest=el[0], el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,) + r[i]))] + r[i+1:]
        yield [(f,)] + r
def uparts(el, size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:], size-1):
        blk=(f,) + rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem, size): yield [tuple(sorted(blk))] + t
def meet(P, Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x, (p, q) in c.items(): g.setdefault((p, q), []).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P, s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P, s): return set(map(frozenset, P)) == set(map(frozenset, imgP(P, s)))
def lab(P, n):
    L=[0]*n
    for i, b in enumerate(P):
        for x in b: L[x]=i
    return L

# ----- reader records -----
def record_R1(t, L, P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t, L, T):
    n=len(t); out=[]
    for x in range(n):
        s=[]; y=x
        for _ in range(T): s.append(L[y]); y=t[y]
        out.append(tuple(s))
    return tuple(sorted(out))

T0=time.time()
print("=*74)
print("SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P, k in [([(0,1,2), (3,4,5)], 1), ([ (0,3), (1,4), (2,5)], 1), ([ (0,1), (2,3), (4,5)], 2),
              ([ (0,1,2), (3,4,5)], 3), ([ (0,2,4), (1,3,5)], 1)]:
    s=tuple((x+k)%n for x in range(n)); L=lab(P, n)
    cons=[t for t in allp if record_R1(t, L, P)==record_R1(s, L, P)]
    I=dcoiset(s, blockgens(P, n))
    cf=young(P)**2//young(meet(P, imgP(P, s)))
    print(f" P={P} sigma=+{k}: |consistent|={len(cons):4d} |KsK|={len(I):4d} "
          f"closed form={cf:4d} match={len(cons)==len(I)==cf}")

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

print("\n"+"="*74)
print("SECTION 4.  the ladder  G <= R3 <= R2 <= R1  (class reps x all set partitions)")
print("  n  configs  |R2|<|R1|  |R3|<|R2|  R3==G  R1==G")
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    tot=s21=s32=eqG=r1G=0; mult=Counter()
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens)
            pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(k,comp(s,inv(k))) for k in K)
            assert G<=R3<=R2<=R1
            tot+=1
            s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
            eqG+= R3==G; r1G+= R1==G
            mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
    print(f"  {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
    if n==6:
        print(f"  |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"="*74)
print("SECTION 5.  R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[(log2(young(P)**2//young(meet(P,imgP(P,s))))), aligned(P,s)) for P in
uparts(range(N),bs)]
        rows.sort(); K=log2(young([tuple(range(bs))]*(N//bs)))
        al=[r[0] for r in rows if r[1]]
        ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
        print(f"  sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"="*74)
print("SECTION 6.  passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"  <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b) "
f"consistent {h:>9} ({log2(h):6.3f} b)  ratio {a/h:.1f}x")
print(f"  transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7.  joint reader census S_4..S_7, and the extremal family")
print("  n  configs  gain  max excess  aligned among gainers")

```

```

for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:
            I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
            e=log2(len(I))-log2(M)-d; tot+=1
            if e>1e-9:
                sup+=1
                if aligned(P,s): alsup+=1
                mx=max(mx,e)
        print(f"    {n} {tot:8d} {sup:6d} {mx:+.4f} {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"    n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
          f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

print("\n"+"="*74)
print("SECTION 8. families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"    n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
    three=[P for P in setparts(range(n)) if len(P)==3]
    kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    cos=[dcoset(s,blockgens(P,n)) for P in props]
    best=None
    for m in (1,2):
        for cb in combinations(range(len(props)),m):
            I=set(allp)
            for i in cb: I&=cos[i]
            if len(I)==1: best=(m,[props[i] for i in cb]); break
        if best: break
    print(f"    n={n}: 3-block readers breaking the tie: {kills}/{len(three)}; "
          f"minimum proper partitions for exact recovery = {best[0]}")
    print(f"    witness: {best[1]}")

# ===== additional sections for the expanded paper =====
def contingency(P,s):
    m=len(P); idx={}
    for i,b in enumerate(P):
        for x in b: idx[x]=i

```

```

    N=[[0]*m for _ in range(m)]
    for i,b in enumerate(P):
        for x in b: N[i][idx[s[x]]]+=1
    return tuple(tuple(r) for r in N)
def prodfac(N):
    r=1
    for row in N:
        for v in row: r*=factorial(v)
    return r
def undirected(s): return frozenset(frozenset((x,s[x])) for x in range(len(s)))
def necklaces(k):
    return {min(tuple(w[i:]+w[:i]) for i in range(k)) for w in itertools.product((0,1),repeat=k)}
def bracelets(k):
    out=set()
    for w in itertools.product((0,1),repeat=k):
        r=tuple(reversed(w))
        out.add(min([tuple(w[i:]+w[:i]) for i in range(k)]+[tuple(r[i:]+r[:i]) for i in range(k)]))
    return out
def twoblocks(n):
    out=set()
    for kk in range(1,n):
        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]

print("\n"+"="*74)
print("SECTION 3 / 4A. the record IS the contingency table; product formula")
ok=True; checked=0
for n in (4,5):
    for P in setparts(range(n)):
        base_=1
        for b in P: base_*=factorial(len(b))
        for s in permutations(range(n)):
            N=contingency(P,s)
            if base_*base_//prodfac(N) != len(dcoset(s,blockgens(P,n))): ok=False
            checked+=1
print(f" |KsK| = (prod|B|!)^2 / prod N_ij! in all {checked} pairs at n=4,5 : {ok}")
n=6; s=tuple((x+1)%6 for x in range(6))
for P,name in [([(0,1,2),(3,4,5)],"A"),([(0,2,4),(1,3,5)],"B")]:
    N=contingency(P,s); K=1
    for b in P: K*=factorial(len(b))
    print(f" worked example reader {name}: N={N} |K|={K} |KsK|={K*K//prodfac(N)}")

print("\n"+"="*74)
print("SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric")
bad=tot=sym2=tot2=0
for n in (4,5,6):
    for P in setparts(range(n)):
        for s in permutations(range(n)):
            N=contingency(P,s); m=len(N)
            symm=all(N[i][j]==N[j][i] for i in range(m) for j in range(m))
            if symm != (inv(s) in dcoset(s,blockgens(P,n))): bad+=1
            tot+=1
            if m<=2:
                tot2+=1; sym2+= symm
print(f" {tot} triples at n=4,5,6 : counterexamples = {bad}")
print(f" of {tot2} triples with |P|<=2, table symmetric in {sym2} (100% expected)")

print("\n"+"="*74)
print("SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c")
for n in (4,5,6,7):
    allp=list(permutations(range(n))); tb=twoblocks(n); allok=True

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

    for lam in parts_of(n):
        s=rep_of(lam,n); cap=set(allp)
        for P in tb: cap&=dcoset(s,blockgens(P,n))
        c=sum(1 for L in lam if L>=3)
        allok &= (len(cap)==2**c) and (cap=={t for t in allp if undirected(t)==undirected(s)})
    print(f"    n={n} ({{len(tb)}} binary readers, {{len(parts_of(n))}} cycle types): |cap| = 2^c and cap
= orientations : {allok}")

print("\n"+"="*74)
print("SECTION 10. handedness threshold")
for n in (5,6,7):
    s=tuple((x+1)%n for x in range(n)); T=order(s); allsym=True
    for P in twoblocks(n):
        q=s
        for t in range(T):
            N=contingency(P,q)
            if N[0][1]!=N[1][0]: allsym=False
            q=comp(s,q)
    print(f"    n={n}: N(P,sigma^t) symmetric for every power and every binary reader: {allsym}")
print("    k necklaces bracelets chiral? R3 binary family on a k-cycle")
for kk in range(3,8):
    n=kk; allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n):
        L=[0]*n
        for i,b in enumerate(P):
            for x in b: L[x]=i
        tgt=itinerary(s,L,2*n)
        cap&={t for t in allp if itinerary(t,L,2*n)==tgt}
    Nn=len(necklaces(kk)); Bb=len(bracelets(kk))
    print(f"    {kk}    {Nn:8d}    {Bb:9d}    {str(Nn>Bb):7s}    |cap|={{len(cap)}}    exact={{cap=={s}}}")

print("\n"+"="*74)
print("SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy")
for n in (5,6):
    allp=list(permutations(range(n)))
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    mins=set(); dens=[]
    for lam in parts_of(n):
        s=rep_of(lam,n)
        cos=[dcoset(s,blockgens(P,n)) for P in props]
        singles=sum(1 for c in cos if len(c)==1)
        good=tot=0
        for i in range(len(props)):
            for j in range(i+1,len(props)):
                tot+=1; good+= len(cos[i]&cos[j])==1
            mins.add((singles==0, good>0)); dens.append((lam,good,tot))
    print(f"    n={n}: single reader never suffices and some pair always does:
{mins=={(True,True)}}")
    print(f"    pair success rate by cycle type: " + ", ".join(f"{{l}}:{{100*g/t:.1f}}%" for l,g,t
in dens[:4]) + " ...")
    for m in (2,3,4):
        fam=[P for P in setparts(range(n)) if len(P)==m]
        exact=all(len({t for t in allp} & set.intersection(*[dcoset(rep_of(lam,n),blockgens(P,n))
for P in fam]))==1
        for lam in parts_of(n):
            print(f"    complete {{m}}-block family ({{len(fam):3d}} readers): exact for every cycle
type = {exact}")
print(f"\nall sections reproduced in {time.time()-T0:.0f}s")

print("\n"+"="*74)
print("SUPERSEDED (kept as Problem-6 data): equal vs unequal K-class sizes inside R3")
print("    n configs Case A Case B |R3|=|K| in B divisibility failures")

```

```

for n in (4,5,6):
    allP=list(setparts(range(n))); A=B=Bok=viol=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            rem=set(R3); sizes=[]
            while rem:
                t=next(iter(rem))
                cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
                sizes.append(len(cl)); rem-=cl
            if len(set(sizes))==1 and sizes[0]==len(G): A+=1
            else:
                B+=1
                if len(R3)==len(K): Bok+=1
            if len(R3)%len(G)!=0: viol+=1
    print(f"  {n} {A+B:8d} {A:8d} {B:8d} {Bok:13d} {viol:20d}")

print("\n"+"="*74)
print("THEOREM 5.1. |R3| = |K| for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4")
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def alignedP(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
for n in (4,5,6):
    allP=list(setparts(range(n)))
    tot=idok=termok=c2=c4=triv=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            H=2*n; tgt=itinerary(s,L,H)
            R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            CK=set(kk for kk in K if comp(kk,s)==comp(s,kk))
            words=Counter()
            for x in range(n):
                w=[]; y=x
                for _ in range(H): w.append(L[y]); y=s[y]
                words[tuple(w)]+=1
            assign=1
            for i,b in enumerate(P):

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

        den=1
        for w,m in words.items():
            if w[0]==i: den*=factorial(m)
        assign*= factorial(len(b))/den
    bij=1
    for w,m in words.items(): bij*=factorial(m)
    tot+=1
    idok += len(R3)==len(K)
    termok += assign*bij==len(K)==len(R3)
    c2 += (R3==G)==(len(CK)==1)
    a=alignedP(P,s)
    c4 += (a==(len(R1)==len(K))==(R2==R1)==(R3==R1))
    triv += len(CK)==1
    print(f"    n={n}: {tot:5d} configs    |R3|=|K|: {idok}    proof terms match: {termok}"
          f"    Cor 5.3 holds: {c2}    Prop 5.4 holds: {c4}    C_K(sigma) trivial: {triv}"
          f"({100*triv/tot:.1f}%)")

print("\n"+"="*74)
print("THEOREM 5.5 / COROLLARY 5.6.  structure of R3 from the word-class shift orbits")
def conj_classes_of_Sm(m):
    out=[]
    def pr(n,mx):
        if n==0: yield (); return
        for kk in range(min(n,mx),0,-1):
            for t in pr(n-kk,kk): yield (kk,)+t
    for lam in pr(m,m):
        c=Counter(lam); d=1
        for l,mu in c.items(): d*=(l**mu)*factorial(mu)
        out.append(factorial(m)//d)
    return out
def pcount(m,mx=None):
    mx=m if mx is None else mx
    if m==0: return 1
    return sum(pcount(m-kk,kk) for kk in range(min(m,mx),0,-1))
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=sz=cnt=corr=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            H=2*n; tgt=itinerary(s,L,H)
            R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
            rem=set(R3); obs=[]
            while rem:
                t=next(iter(rem))
                cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
                obs.append(len(cl)); rem-=cl
            word={}
            for x in range(n):
                w=[]; y=x
                for _ in range(H): w.append(L[y]); y=s[y]
                word[x]=tuple(w)
            cls={}

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

for x in range(n): cls.setdefault(word[x],[]).append(x)
nxt={w: word[s[cls[w][0]]] for w in cls}
seen=set(); ms=[]
for w in cls:
    if w in seen: continue
    y=w
    while y not in seen: seen.add(y); y=nxt[y]
    ms.append(len(cls[w]))
pred=[]
for combo in itertools.product(*[conj_classes_of_Sm(m) for m in ms]):
    cent=1
    for cs,m in zip(combo,ms): cent*= factorial(m)//cs
    pred.append(len(K)//cent)
tot+=1
sz += sorted(obs)==sorted(pred)
npred=1
for m in ms: npred*=pcount(m)
cnt += len(obs)==npred
corr += (len(set(obs))==1)==(max(ms)<=2)
print(f"    n={n}: {tot:5d} configs    class-size multiset predicted: {sz}    class count = prod
p(m): {cnt}"
      f"    equal sizes iff all word-classes <= 2: {corr}")

print("\n"+"="*74)
print("PROOF STEPS FOR THEOREM 5.5 (holonomy argument)")
print("    S1: K transitive on assignments, stabiliser = prod of symmetric groups on word-classes")
print("    S2: C_K(tau) <= Stab(A_tau)    (tau's own word-classes, not sigma's)")
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=s1=s2=s2wrong=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                    q=comp(t,q)
                if ok: R2.add(t)
            H=2*n; tgt=itinerary(s,L,H)
            R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
            def words_of(t):
                d={}
                for x in range(n):
                    w=[]; y=x
                    for _ in range(H): w.append(L[y]); y=t[y]
                    d[x]=tuple(w)
                return d
            ws=words_of(s); cls={}
            for x in range(n): cls.setdefault(ws[x],[]).append(x)
            fib=[kk for kk in K if all(ws[kk[x]]==ws[x] for x in range(n))]
            exp=1
            for w,v in cls.items(): exp*=factorial(len(v))
            assign=1
            for i,b in enumerate(P):
                den=1
                for w,v in cls.items():
                    if w[0]==i: den*=factorial(len(v))
                assign*=factorial(len(b))//den

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

    tot+=1
    if len(fib)==exp and len(K)//len(fib)==assign: s1+=1
    okc=okw=True
    for t in R3:
        wt=words_of(t)
        for kk in K:
            if comp(kk,t)==comp(t,kk):
                if any(wt[kk[x]]!=wt[x] for x in range(n)): okc=False
                if any(ws[kk[x]]!=ws[x] for x in range(n)): okw=False
    s2+=okc; s2wrong+= not okw
    print(f"    n={n}: {tot:5d} configs    S1: {s1}    S2 (tau's classes): {s2}"
          f"    configs where the sigma-version of S2 fails: {s2wrong}")

print("\n"+"="*74)
print("SECTION 6A.  the sampling-horizon ladder and the critical horizon T*")
def itinT(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        w=[]; y=x
        for _ in range(T): w.append(L[y]); y=t[y]
        out.append(tuple(w))
    return tuple(sorted(out))
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    a=b=c=tot=0; byblocks={}
    for lam in parts_of(n):
        s=rep_of(lam,n)
        for P in allP:
            K=young_group(P,n); L=lab(P,n)
            R1=dcoset(s,blockgens(P,n))
            sets=[set(t for t in allp if itinT(t,L,T)==itinT(s,L,T)) for T in range(1,3*n+1)]
            tot+=1
            a += len(sets[0])==factorial(n)
            b += sets[1]==R1
            c += all(sets[i+1]<=sets[i] for i in range(len(sets)-1)) and len(sets[-1])==len(K)
            fin=sets[-1]
            ts=next(T for T in range(1,3*n+1) if sets[T-1]==fin)
            byblocks.setdefault(len(P),[]).append(ts)
    law=all(max(v)==(n-m+2 if m>=2 else 1) for m,v in byblocks.items())
    print(f"    n={n}: {tot:5d} configs    R^(1)=Sym(X): {a}    R^(2)=KsK: {b}    monotone & saturates
at |K|: {c}")
    print(f"        max T* by blocks m: "+", ".join(f"{m}:{max(v)}" for m,v in
sorted(byblocks.items())))
    +f"    matches n-m+2 (m>=2): {law}"
26. =====
SECTION 3.  R1 consistent set = the double coset KsK    (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma=+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma=+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma=+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma=+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma=+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
=====
SECTION 4.  the ladder G <= R3 <= R2 <= R1    (class reps x all set partitions)
n   configs   |R2|<|R1|   |R3|<|R2|   R3==G   R1==G
4    75        40        3    39 (52.0%)    5
5   364       262       18   201 (55.2%)    7
6  2233      1838      132  1205 (54.0%)   11
    |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41, 4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72,
16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1}    (no non-integer ratios: True)
=====
SECTION 5.  R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)

```

```

sigma==+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma==+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma==+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma==+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma==+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma==+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma==+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma==+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma==+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma==+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma==+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma==+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
<+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
<+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio
6.0x
<+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 ( 7.755 b) ratio
16.0x
transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
n configs gain max excess aligned among gainers
4 75 2 +0.5850 0
5 364 21 +1.0000 0
6 2233 148 +1.5850 0
7 13155 929 +2.3219 0
extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4
n=6: |KsK| 288/288 |M| 12/12 gain 3.000/3.000 excess +1.5850 b
n=7: |KsK| 2400/2400 |M| 20/20 gain 5.000/5.000 excess +2.3219 b
n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500 excess +2.9069 b
n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor
n=5: complete binary family 15 readers (= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}:
True
n=6: complete binary family 31 readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}:
True
n=7: complete binary family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}:
True
n=5: 3-block readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2
witness: [[(0,), (1,), (2, 3, 4)], [(0, 2), (3,), (1, 4)]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[(0,), (1,), (2, 3, 4, 5)], [(0, 2), (3,), (4,), (1, 5)]]

=====
SECTION 3 / 4A. the record IS the contingency table; product formula
|KsK| = (prod|B|!)^2 / prod N_ij! in all 6600 pairs at n=4,5 : True
worked example reader A: N=((2, 1), (1, 2)) |K|=36 |KsK|=324
worked example reader B: N=((0, 3), (3, 0)) |K|=36 |KsK|=36

=====
SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric
152760 triples at n=4,5,6 : counterexamples = 0
of 25152 triples with |P|<=2, table symmetric in 25152 (100% expected)

=====
SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c
n=4 (7 binary readers, 5 cycle types): |cap| = 2^c and cap = orientations : True
n=5 (15 binary readers, 7 cycle types): |cap| = 2^c and cap = orientations : True

```

n=6 (31 binary readers, 11 cycle types): $|cap| = 2^c$ and $cap = orientations : True$
 n=7 (63 binary readers, 15 cycle types): $|cap| = 2^c$ and $cap = orientations : True$

SECTION 10. handedness threshold

n=5: N(P,sigma^t) symmetric for every power and every binary reader: True
 n=6: N(P,sigma^t) symmetric for every power and every binary reader: True
 n=7: N(P,sigma^t) symmetric for every power and every binary reader: True

k	necklaces	bracelets	chiral?	R3 binary family on a k-cycle
3	4	4	False	$ cap =2$ exact=False
4	6	6	False	$ cap =2$ exact=False
5	8	8	False	$ cap =2$ exact=False
6	14	13	True	$ cap =1$ exact=True
7	20	18	True	$ cap =1$ exact=True

SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy

n=5: single reader never suffices and some pair always does: True
 pair success rate by cycle type: (5,):14.7%, (4, 1):14.7%, (3, 2):17.1%, (3, 1, 1):15.9% ...
 complete 2-block family (15 readers): exact for every cycle type = False
 complete 3-block family (25 readers): exact for every cycle type = True
 complete 4-block family (10 readers): exact for every cycle type = True

n=6: single reader never suffices and some pair always does: True
 pair success rate by cycle type: (6,):11.9%, (5, 1):11.0%, (4, 2):13.1%, (4, 1, 1):12.2% ...
 complete 2-block family (31 readers): exact for every cycle type = False
 complete 3-block family (90 readers): exact for every cycle type = True
 complete 4-block family (65 readers): exact for every cycle type = True

all sections reproduced in 125s

SUPERSEDED (kept as Problem-6 data): equal vs unequal K-class sizes inside R3

n	configs	Case A	Case B	$ R3 = K $ in B	divisibility failures
4	75	63	12	12	0
5	364	301	63	63	0
6	2233	1869	364	364	0

THEOREM 5.1. $|R3| = |K|$ for every sigma and P; Corollaries 5.2, 5.3; Prop 5.4

n=4: 75 configs $|R3|=|K|$: 75 proof terms match: 75 Cor 5.3 holds: 75 Prop 5.4 holds: 75
 C_K(sigma) trivial: 39 (52.0%)

n=5: 364 configs $|R3|=|K|$: 364 proof terms match: 364 Cor 5.3 holds: 364 Prop 5.4 holds: 364
 C_K(sigma) trivial: 201 (55.2%)

n=6: 2233 configs $|R3|=|K|$: 2233 proof terms match: 2233 Cor 5.3 holds: 2233 Prop 5.4 holds: 2233
 C_K(sigma) trivial: 1205 (54.0%)

THEOREM 5.5 / COROLLARY 5.6. structure of R3 from the word-class shift orbits

n=4: 75 configs class-size multiset predicted: 75 class count = prod p(m): 75 equal sizes iff all word-classes ≤ 2 : 75

n=5: 364 configs class-size multiset predicted: 364 class count = prod p(m): 364 equal sizes iff all word-classes ≤ 2 : 364

n=6: 2233 configs class-size multiset predicted: 2233 class count = prod p(m): 2233 equal sizes iff all word-classes ≤ 2 : 2233

PROOF STEPS FOR THEOREM 5.5 (holonomy argument)

S1: K transitive on assignments, stabiliser = prod of symmetric groups on word-classes
 S2: $C_K(\tau) \leq \text{Stab}(A_\tau)$ (τ 's own word-classes, not sigma's)

n=4: 75 configs S1: 75 S2 (τ 's classes): 75 configs where the sigma-version of S2 fails: 6

n=5: 364 configs S1: 364 S2 (τ 's classes): 364 configs where the sigma-version of S2 fails: 52

```

n=6: 2233 configs   S1: 2233   S2 (tau's classes): 2233   configs where the sigma-version of S2
fails: 478

```

```

=====

```

```

SECTION 6A. the sampling-horizon ladder and the critical horizon T*

```

```

n=4:   75 configs   R^(1)=Sym(X): 75   R^(2)=KsK: 75   monotone & saturates at |K|: 75
      max T* by blocks m: 1:1, 2:4, 3:3, 4:2   matches n-m+2 (m>=2): True
n=5:  364 configs   R^(1)=Sym(X): 364   R^(2)=KsK: 364   monotone & saturates at |K|: 364
      max T* by blocks m: 1:1, 2:5, 3:4, 4:3, 5:2   matches n-m+2 (m>=2): True
n=6: 2233 configs   R^(1)=Sym(X): 2233   R^(2)=KsK: 2233   monotone & saturates at |K|: 2233
      max T* by blocks m: 1:1, 2:6, 3:5, 4:4, 5:3, 6:2   matches n-m+2 (m>=2): True

```

```

27. """Companion verification for: What a Reader Recovers.
Reproduces every number in the paper. No randomness; every census stated as exhaustive enumerates
its full domain. Runtime ~2 min on one core.
"""

```

```

28.

```

```

29. import itertools, time
from math import factorial, log2
from itertools import permutations, combinations
from collections import Counter, deque

# ----- permutation utilities -----
def comp(a,b): return tuple(a[b[i]] for i in range(len(b)))
def inv(p):
    q=[0]*len(p)
    for i,v in enumerate(p): q[v]=i
    return tuple(q)
def ctype(p):
    n=len(p); seen=[False]*n; c=[]
    for i in range(n):
        if seen[i]: continue
        l=0; j=i
        while not seen[j]: seen[j]=True; j=p[j]; l+=1
        c.append(l)
    return tuple(sorted(c,reverse=True))
def order(p):
    n=len(p); e=tuple(range(n)); q=p; k=1
    while q!=e: q=comp(p,q); k+=1
    return k
def class_size(lam,n):
    cnt=Counter(lam); d=1
    for l,m in cnt.items(): d*=(1**m)*factorial(m)
    return factorial(n)//d
def rep_of(lam,n):
    p=[0]*n; i=0
    for L in lam:
        for j in range(L): p[i+j]=i+(j+1)%L
        i+=L
    return tuple(p)
def parts_of(n):
    def go(n,mx):
        if n==0: yield (); return
        for k in range(min(n,mx),0,-1):
            for t in go(n-k,k): yield (k,)+t
    return list(go(n,n))

# ----- partitions and Young subgroups -----
def young(P):
    r=1
    for b in P: r*=factorial(len(b))
    return r
def young_group(P,n):
    B=[list(b) for b in P]; out=[]

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

    for cb in itertools.product(*[permutations(b) for b in B]):
        q=[0]*n
        for b,img in zip(B,cb):
            for s,d in zip(b,img): q[s]=d
            out.append(tuple(q))
    return out
def blockgens(P,n):
    g=[]
    for b in P:
        b=sorted(b)
        for x,y in zip(b,b[1:]):
            q=list(range(n)); q[x],q[y]=y,x; g.append(tuple(q))
    return g
def dcset(s,gens):
    """KSK by BFS under left/right multiplication by block transpositions."""
    I={s}; q=deque([s])
    while q:
        p=q.popleft()
        for g in gens:
            for r in (comp(g,p), comp(p,g)):
                if r not in I: I.add(r); q.append(r)
    return I
def setparts(el):
    el=list(el)
    if not el: yield []; return
    f,rest=el[0],el[1:]
    for r in setparts(rest):
        for i in range(len(r)): yield r[:i]+[tuple(sorted((f,)+r[i]))]+r[i+1:]
        yield [(f,)]+r
def uparts(el,size):
    el=list(el)
    if not el: yield []; return
    f=el[0]
    for rest in combinations(el[1:],size-1):
        blk=(f,)+rest; rem=[e for e in el[1:] if e not in rest]
        for t in uparts(rem,size): yield [tuple(sorted(blk))]+t
def meet(P,Q):
    c={}
    for b in P:
        for x in b: c[x]=[frozenset(b)]
    for b in Q:
        for x in b: c[x].append(frozenset(b))
    g={}
    for x,(p,q) in c.items(): g.setdefault((p,q),[]).append(x)
    return [tuple(sorted(v)) for v in g.values()]
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def aligned(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
def lab(P,n):
    L=[0]*n
    for i,b in enumerate(P):
        for x in b: L[x]=i
    return L

# ----- reader records -----
def record_R1(t,L,P):
    return tuple(tuple(sorted(Counter(L[t[x]] for x in b).items())) for b in P)
def itinerary(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        s=[]; y=x
        for _ in range(T): s.append(L[y]); y=t[y]
        out.append(tuple(s))
    return tuple(sorted(out))

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

T0=time.time()
print("=*74)
print("SECTION 3. R1 consistent set = the double coset KsK (brute force, S_6)")
n=6; allp=list(permutations(range(n)))
for P,k in [((0,1,2),(3,4,5)),1],((0,3),(1,4),(2,5)),1],((0,1),(2,3),(4,5)),2],
            ((0,1,2),(3,4,5)),3],((0,2,4),(1,3,5)),1]):
    s=tuple((x+k)%n for x in range(n)); L=lab(P,n)
    cons=[t for t in allp if record_R1(t,L,P)==record_R1(s,L,P)]
    I=dcoset(s,blockgens(P,n))
    cf=young(P)**2//young(meet(P,imgP(P,s)))
    print(f" P={P} sigma={k}: |consistent|={len(cons):4d} |KsK|={len(I):4d} "
          f"closed form={cf:4d} match={len(cons)==len(I)==cf}")

print("\n"+"=*74)
print("SECTION 4. the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)")
print(" n configs |R2|<|R1| |R3|<|R2| R3==G R1==G")
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    tot=s21=s32=eqG=r1G=0; mult=Counter()
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens)
            pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(k,comp(s,inv(k))) for k in K)
            assert G<=R3<=R2<=R1
            tot+=1
            s21+= len(R2)<len(R1); s32+= len(R3)<len(R2)
            eqG+= R3==G; r1G+= R1==G
            mult[len(R3)//len(G) if len(R3)%len(G)==0 else -1]+=1
    print(f" {n} {tot:8d} {s21:10d} {s32:11d} {eqG:5d} ({100*eqG/tot:4.1f}%) {r1G:8d}")
    if n==6:
        print(f" |R3|/|G| multiplicities: {dict(sorted(mult.items()))} (no non-integer
ratios: {-1 not in mult})")

print("\n"+"=*74)
print("SECTION 5. R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)")
N=12
for k in (1,2,3):
    s=[(x+k)%N for x in range(N)]
    for bs in (6,4,3,2):
        rows=[(log2(young(P)**2//young(meet(P,imgP(P,s))))), aligned(P,s)) for P in
uparts(range(N),bs)]
        rows.sort(); K=log2(young([tuple(range(bs))* (N//bs)))
        al=[r[0] for r in rows if r[1]]
        ok = abs(rows[0][0]-K)<1e-9 and min(al)==max(al)==rows[0][0]
        print(f" sigma={k} labels={N//bs}: floor={K:7.3f} min={rows[0][0]:7.3f} "
              f"median={rows[len(rows)//2][0]:7.3f} worst={rows[-1][0]:7.3f} aligned-attains={ok}")

print("\n"+"=*74)

```

```

print("SECTION 6.  passive observation: consistent set vs the group-based figure (Z/12)")
import math
for k in (1,2,3):
    G=12//math.gcd(12,k); sizes=[G]*(12//G); a=1; h=1
    for z in sizes: a*=factorial(z); h*=factorial(z-1)
    a//=G
    print(f"    <+{k}> orbits {sizes}: group-based {a:>9} ({log2(a):6.3f} b)    "
          f"consistent {h:>9} ({log2(h):6.3f} b)    ratio {a/h:.1f}x")
print(f"    transitive coincidence: 12!/12 = 11! = {factorial(12)//12 == factorial(11)}")

print("\n"+"="*74)
print("SECTION 7.  joint reader census S_4..S_7, and the extremal family")
print("    n configs  gain  max excess  aligned among gainers")
for n in (4,5,6,7):
    allP=list(setparts(range(n))); tot=sup=alsup=0; mx=-9
    for lam in parts_of(n):
        s=rep_of(lam,n); Cl=class_size(lam,n); d=log2(factorial(n)/Cl)
        for P in allP:
            I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
            e=log2(len(I))-log2(M)-d; tot+=1
            if e>1e-9:
                sup+=1
                if aligned(P,s): alsup+=1
            mx=max(mx,e)
    print(f"    {n} {tot:8d} {sup:6d}    {mx:+.4f}    {alsup}")
print("    extremal family lambda=(4,1^(n-4)), P={1,3}|rest,  gain=(n-2)(n-3)/4")
for n in (6,7,8,9):
    lam=(4,)+(1,)*(n-4); s=rep_of(lam,n); Cl=class_size(lam,n)
    P=[(1,3),tuple(x for x in range(n) if x not in (1,3))]
    I=dcoset(s,blockgens(P,n)); M=sum(1 for p in I if ctype(p)==lam)
    R=(len(I)*Cl/factorial(n))/M
    print(f"    n={n}: |KsK| {factorial(n-2)**2//factorial(n-4):>7}/{len(I):<7} "
          f"|M| {(n-2)*(n-3):>3}/{M:<3} gain {(n-2)*(n-3)/4:7.3f}/{R:<7.3f} excess {log2(R):+.4f}
b")

print("\n"+"="*74)
print("SECTION 8.  families of readers, and the orientation floor")
def twoblocks(n):
    out=set()
    for k in range(1,n):
        for A in combinations(range(n),k):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]
for n in (5,6,7):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    tb=twoblocks(n); cap=set(allp)
    for P in tb: cap&=dcoset(s,blockgens(P,n))
    print(f"    n={n}: complete binary family {len(tb)} readers (= 2^(n-1)-1 = {2**(n-1)-1}) "
          f"-> {len(cap)} candidates; == {{sigma, sigma^-1}}: {cap=={s,inv(s)}}")
for n in (5,6):
    allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n): cap&=dcoset(s,blockgens(P,n))
    three=[P for P in setparts(range(n)) if len(P)==3]
    kills=sum(1 for P in three if len(cap & dcoset(s,blockgens(P,n)))==1)
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    cos=[dcoset(s,blockgens(P,n)) for P in props]
    best=None
    for m in (1,2):
        for cb in combinations(range(len(props)),m):
            I=set(allp)
            for i in cb: I&=cos[i]

```

```

        if len(I)==1: best=(m,[props[i] for i in cb]); break
    if best: break
print(f"    n={n}: 3-block readers breaking the tie: {kills}/{len(three)};  "
      f"minimum proper partitions for exact recovery = {best[0]}")
print(f"        witness: {best[1]}")

# ===== additional sections for the expanded paper =====
def contingency(P,s):
    m=len(P); idx={}
    for i,b in enumerate(P):
        for x in b: idx[x]=i
    N=[[0]*m for _ in range(m)]
    for i,b in enumerate(P):
        for x in b: N[i][idx[s[x]]]+=1
    return tuple(tuple(r) for r in N)
def prodfac(N):
    r=1
    for row in N:
        for v in row: r*=factorial(v)
    return r
def undirected(s): return frozenset(frozenset((x,s[x])) for x in range(len(s)))
def necklaces(k):
    return {min(tuple(w[i:]+w[:i]) for i in range(k)) for w in itertools.product((0,1),repeat=k)}
def bracelets(k):
    out=set()
    for w in itertools.product((0,1),repeat=k):
        r=tuple(reversed(w))
        out.add(min([tuple(w[i:]+w[:i]) for i in range(k)]+[tuple(r[i:]+r[:i]) for i in range(k)]))
    return out
def twoblocks(n):
    out=set()
    for kk in range(1,n):
        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
    return [sorted(map(tuple,f)) for f in out]

print("\n"+"="*74)
print("SECTION 3 / 4A.  the record IS the contingency table; product formula")
ok=True; checked=0
for n in (4,5):
    for P in setparts(range(n)):
        base_=1
        for b in P: base_*=factorial(len(b))
        for s in permutations(range(n)):
            N=contingency(P,s)
            if base_*base_//prodfac(N) != len(dcoset(s,blockgens(P,n))): ok=False
            checked+=1
print(f"    |KsK| = (prod|B|!)^2 / prod N_ij!  in all {checked} pairs at n=4,5 : {ok}")
n=6; s=tuple((x+1)%6 for x in range(6))
for P,name in [((0,1,2),(3,4,5)),"A"],((0,2,4),(1,3,5)),"B"]]:
    N=contingency(P,s); K=1
    for b in P: K*=factorial(len(b))
    print(f"    worked example reader {name}: N={N}  |K|={K}  |KsK|={K*K//prodfac(N)}")

print("\n"+"="*74)
print("SECTION 4.  inversion criterion:  sigma^-1 in KsK  <=>  N(P,sigma) symmetric")
bad=tot=sym2=tot2=0
for n in (4,5,6):
    for P in setparts(range(n)):
        for s in permutations(range(n)):

```

```

        N=contingency(P,s); m=len(N)
        symm=all(N[i][j]==N[j][i] for i in range(m) for j in range(m))
        if symm != (inv(s) in dcoset(s,blockgens(P,n))): bad+=1
        tot+=1
        if m<=2:
            tot2+=1; sym2+= symm
print(f" {tot} triples at n=4,5,6 : counterexamples = {bad}")
print(f" of {tot2} triples with |P|<=2, table symmetric in {sym2} (100% expected)")

print("\n"+"="*74)
print("SECTION 9. complete binary family recovers the undirected graph; |cap| = 2^c")
for n in (4,5,6,7):
    allp=list(permutations(range(n))); tb=twoblocks(n); allok=True
    for lam in parts_of(n):
        s=rep_of(lam,n); cap=set(allp)
        for P in tb: cap&=dcoset(s,blockgens(P,n))
        c=sum(1 for L in lam if L>=3)
        allok &= (len(cap)==2**c) and (cap=={t for t in allp if undirected(t)==undirected(s)})
    print(f" n={n} ({len(tb)} binary readers, {len(parts_of(n))} cycle types): |cap| = 2^c and cap
= orientations : {allok}")

print("\n"+"="*74)
print("SECTION 10. handedness threshold")
for n in (5,6,7):
    s=tuple((x+1)%n for x in range(n)); T=order(s); allsym=True
    for P in twoblocks(n):
        q=s
        for t in range(T):
            N=contingency(P,q)
            if N[0][1]!=N[1][0]: allsym=False
        q=comp(s,q)
    print(f" n={n}: N(P,sigma^t) symmetric for every power and every binary reader: {allsym}")
print(" k necklaces bracelets chiral? R3 binary family on a k-cycle")
for kk in range(3,8):
    n=kk; allp=list(permutations(range(n))); s=tuple((x+1)%n for x in range(n))
    cap=set(allp)
    for P in twoblocks(n):
        L=[0]*n
        for i,b in enumerate(P):
            for x in b: L[x]=i
        tgt=itinerary(s,L,2*n)
        cap&={t for t in allp if itinerary(t,L,2*n)==tgt}
    Nn=len(necklaces(kk)); Bb=len(bracelets(kk))
    print(f" {kk} {Nn:8d} {Bb:9d} {str(Nn>Bb):7s} |cap|={len(cap)} exact={cap=={s}}")

print("\n"+"="*74)
print("SECTION 11 / 11A. families: minimum size, pair density, and the m-block dichotomy")
for n in (5,6):
    allp=list(permutations(range(n)))
    props=[P for P in setparts(range(n)) if 1<len(P)<n]
    mins=set(); dens=[]
    for lam in parts_of(n):
        s=rep_of(lam,n)
        cos=[dcoset(s,blockgens(P,n)) for P in props]
        singles=sum(1 for c in cos if len(c)==1)
        good=tot=0
        for i in range(len(props)):
            for j in range(i+1,len(props)):
                tot+=1; good+= len(cos[i]&cos[j])==1
            mins.add((singles==0, good>0)); dens.append((lam,good,tot))
    print(f" n={n}: single reader never suffices and some pair always does:
{mins=={(True,True)}}")
    print(f" pair success rate by cycle type: " + ", ".join(f"{l}:{100*g/t:.1f}%" for l,g,t

```

```

in dens[:4]) + " ...")
    for m in (2,3,4):
        fam=[P for P in setparts(range(n)) if len(P)==m]
        exact=all(len({t for t in allp} & set.intersection(*[dcoset(rep_of(lam,n),blockgens(P,n))
for P in fam]))==1
            for lam in parts_of(n))
        print(f"            complete {m}-block family ({len(fam):3d} readers): exact for every cycle
type = {exact}")
print(f"\nall sections reproduced in {time.time()-T0:.0f}s")

print("\n"+"="*74)
print("SUPERSEDED (kept as Problem-6 data): equal vs unequal K-class sizes inside R3")
print("  n  configs  Case A   Case B   |R3|=|K| in B   divisibility failures")
for n in (4,5,6):
    allP=list(setparts(range(n))); A=B=Bok=viol=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                    q=comp(t,q)
                if ok: R2.add(t)
            tgt=itinerary(s,L,2*n)
            R3=set(t for t in R2 if itinerary(t,L,2*n)==tgt)
            G=set(comp(kk,comp(s,inv(kk))) for kk in K)
            rem=set(R3); sizes=[]
            while rem:
                t=next(iter(rem))
                cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
                sizes.append(len(cl)); rem-=cl
            if len(set(sizes))==1 and sizes[0]==len(G): A+=1
            else:
                B+=1
                if len(R3)==len(K): Bok+=1
                if len(R3)%len(G)!=0: viol+=1
    print(f"  {n}  {A+B:8d}  {A:8d}  {B:8d}  {Bok:13d}  {viol:20d}")

print("\n"+"="*74)
print("THEOREM 5.1.  |R3| = |K| for every sigma and P;  Corollaries 5.2, 5.3;  Prop 5.4")
def imgP(P,s): return [tuple(sorted(s[x] for x in b)) for b in P]
def alignedP(P,s): return set(map(frozenset,P))==set(map(frozenset,imgP(P,s)))
for n in (4,5,6):
    allP=list(setparts(range(n)))
    tot=idok=termok=c2=c4=triv=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break

```

```

        q=comp(t,q)
        if ok: R2.add(t)
H=2*n; tgt=itinerary(s,L,H)
R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
G=set(comp(kk,comp(s,inv(kk))) for kk in K)
CK=set(kk for kk in K if comp(kk,s)==comp(s,kk))
words=Counter()
for x in range(n):
    w=[]; y=x
    for _ in range(H): w.append(L[y]); y=s[y]
    words[tuple(w)]+=1
assign=1
for i,b in enumerate(P):
    den=1
    for w,m in words.items():
        if w[0]==i: den*=factorial(m)
    assign*= factorial(len(b))/den
bij=1
for w,m in words.items(): bij*=factorial(m)
tot+=1
idok += len(R3)==len(K)
termok += assign*bij==len(K)==len(R3)
c2 += (R3==G)==(len(CK)==1)
a=alignedP(P,s)
c4 += (a==(len(R1)==len(K))==(R2==R1)==(R3==R1))
triv += len(CK)==1
print(f"    n={n}: {tot:5d} configs    |R3|=|K|: {idok}    proof terms match: {termok}"
      f"    Cor 5.3 holds: {c2}    Prop 5.4 holds: {c4}    C_K(sigma) trivial: {triv}"
      f"({100*triv/tot:.1f}%)")

print("\n"+"="*74)
print("THEOREM 5.5 / COROLLARY 5.6.  structure of R3 from the word-class shift orbits")
def conj_classes_of_Sm(m):
    out=[]
    def pr(n,mx):
        if n==0: yield (); return
        for kk in range(min(n,mx),0,-1):
            for t in pr(n-kk,kk): yield (kk,)+t
    for lam in pr(m,m):
        c=Counter(lam); d=1
        for l,mu in c.items(): d*=(l**mu)*factorial(mu)
        out.append(factorial(m)//d)
    return out
def pcount(m,mx=None):
    mx=m if mx is None else mx
    if m==0: return 1
    return sum(pcount(m-kk,kk) for kk in range(min(m,mx),0,-1))
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=sz=cnt=corr=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                q=comp(t,q)
                if ok: R2.add(t)

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

H=2*n; tgt=itinerary(s,L,H)
R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
rem=set(R3); obs=[]
while rem:
    t=next(iter(rem))
    cl=set(comp(kk,comp(t,inv(kk))) for kk in K)
    obs.append(len(cl)); rem-=cl
word={}
for x in range(n):
    w=[]; y=x
    for _ in range(H): w.append(L[y]); y=s[y]
    word[x]=tuple(w)
cls={}
for x in range(n): cls.setdefault(word[x],[]).append(x)
nxt={w: word[s[cls[w][0]]] for w in cls}
seen=set(); ms=[]
for w in cls:
    if w in seen: continue
    y=w
    while y not in seen: seen.add(y); y=nxt[y]
    ms.append(len(cls[w]))
pred=[]
for combo in itertools.product(*[conj_classes_of_Sm(m) for m in ms]):
    cent=1
    for cs,m in zip(combo,ms): cent*= factorial(m)//cs
    pred.append(len(K)//cent)
tot+=1
sz += sorted(obs)==sorted(pred)
npred=1
for m in ms: npred*=pcount(m)
cnt += len(obs)==npred
corr += (len(set(obs))==1)==(max(ms)<=2)
print(f"    n={n}: {tot:5d} configs    class-size multiset predicted: {sz}    class count = prod
p(m): {cnt}"
      f"    equal sizes iff all word-classes <= 2: {corr}")

print("\n"+"="*74)
print("PROOF STEPS FOR THEOREM 5.5 (holonomy argument)")
print("    S1: K transitive on assignments, stabiliser = prod of symmetric groups on word-classes")
print("    S2:  $C_K(\tau) \leq \text{Stab}(A_\tau)$  ( $\tau$ 's own word-classes, not  $\sigma$ 's)")
for n in (4,5,6):
    allP=list(setparts(range(n))); tot=s1=s2=s2wrong=0
    for lam in parts_of(n):
        s=rep_of(lam,n); T=order(s)
        pw=[]; q=s
        for _ in range(T): pw.append(q); q=comp(s,q)
        for P in allP:
            K=young_group(P,n); L=lab(P,n); gens=blockgens(P,n)
            R1=dcoset(s,gens); pc=[dcoset(p,gens) for p in pw]
            R2=set()
            for t in R1:
                q=t; ok=True
                for i in range(T):
                    if q not in pc[i]: ok=False; break
                    q=comp(t,q)
                if ok: R2.add(t)
H=2*n; tgt=itinerary(s,L,H)
R3=set(t for t in R2 if itinerary(t,L,H)==tgt)
def words_of(t):
    d={}
    for x in range(n):
        w=[]; y=x
        for _ in range(H): w.append(L[y]); y=t[y]

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

        d[x]=tuple(w)
    return d
ws=words_of(s); cls={}
for x in range(n): cls.setdefault(ws[x],[]).append(x)
fib=[kk for kk in K if all(ws[kk[x]]==ws[x] for x in range(n))]
exp=1
for w,v in cls.items(): exp*=factorial(len(v))
assign=1
for i,b in enumerate(P):
    den=1
    for w,v in cls.items():
        if w[0]==i: den*=factorial(len(v))
    assign*=factorial(len(b))//den
tot+=1
if len(fib)==exp and len(K)//len(fib)==assign: s1+=1
okc=okw=True
for t in R3:
    wt=words_of(t)
    for kk in K:
        if comp(kk,t)==comp(t,kk):
            if any(wt[kk[x]]!=wt[x] for x in range(n)): okc=False
            if any(ws[kk[x]]!=ws[x] for x in range(n)): okw=False
s2+=okc; s2wrong+= not okw
print(f"    n={n}: {tot:5d} configs    S1: {s1}    S2 (tau's classes): {s2}"
      f"    configs where the sigma-version of S2 fails: {s2wrong}")

print("\n"+"="*74)
print("SECTION 6A.  the sampling-horizon ladder and the critical horizon T*")
def itinT(t,L,T):
    n=len(t); out=[]
    for x in range(n):
        w=[]; y=x
        for _ in range(T): w.append(L[y]); y=t[y]
        out.append(tuple(w))
    return tuple(sorted(out))
for n in (4,5,6):
    allp=list(permutations(range(n))); allP=list(setparts(range(n)))
    a=b=c=tot=0; byblocks={}
    for lam in parts_of(n):
        s=rep_of(lam,n)
        for P in allP:
            K=young_group(P,n); L=lab(P,n)
            R1=dcoset(s,blockgens(P,n))
            sets=[set(t for t in allp if itinT(t,L,T)==itinT(s,L,T)) for T in range(1,3*n+1)]
            tot+=1
            a += len(sets[0])==factorial(n)
            b += sets[1]==R1
            c += all(sets[i+1]<=sets[i] for i in range(len(sets)-1)) and len(sets[-1])==len(K)
            fin=sets[-1]
            ts=next(T for T in range(1,3*n+1) if sets[T-1]==fin)
            byblocks.setdefault(len(P),[]).append(ts)
        law=all(max(v)==(n-m+2 if m>=2 else 1) for m,v in byblocks.items())
        print(f"    n={n}: {tot:5d} configs    R^(1)=Sym(X): {a}    R^(2)=KsK: {b}    monotone & saturates
at |K|: {c}")
        print(f"        max T* by blocks m: "+, ".join(f"{m}:{max(v)}" for m,v in
sorted(byblocks.items()))
        +f"    matches n-m+2 (m>=2): {law}")

print("\n"+"="*74)
print("SECTION 11B.  the co-moving reader")
def twoblocks(n):
    out=set()
    for kk in range(1,n):

```

```

        for A in combinations(range(n),kk):
            B=tuple(x for x in range(n) if x not in A)
            out.add(frozenset([A,B]))
        return [sorted(map(tuple,f)) for f in out]
def rec_running(t,L,pi,T):
    n=len(t); pin=inv(pi); pis=[tuple(range(n))]
    for _ in range(T): pis.append(comp(pin,pis[-1]))
    out=[]
    for x in range(n):
        w=[]; y=x
        for s in range(T): w.append(L[pis[s][y]]); y=t[y]
        out.append(tuple(w))
    return tuple(sorted(out))
for n in (4,5):
    allp=list(permutations(range(n))); tb=twoblocks(n); T=4*n
    print(f"      n={n}      cycle type      static binary family      best co-moving binary reader")
    for lam in parts_of(n):
        s=rep_of(lam,n)
        cap=set(allp)
        for P in tb: cap&=dcoset(s,blockgens(P,n))
        best=None
        for P in tb:
            L=[0]*n
            for i,b in enumerate(P):
                for x in b: L[x]=i
            for pi in allp:
                tgt=rec_running(s,L,pi,T)
                c=sum(1 for t in allp if rec_running(t,L,pi,T)==tgt)
                if best is None or c<best: best=c
            note=" <-- co-moving is WORSE (own motion unattributable)" if best>len(cap) else ""
            print(f"                {str(lam):<14}  {len(cap):>6}                {best:>6}{note}")
        s=tuple((x+1)%n for x in range(n)); si=inv(s); w=t_ = 0
        for P in tb:
            L=[0]*n
            for i,b in enumerate(P):
                for x in b: L[x]=i
            for pi in allp:
                t_+=1
                w += rec_running(s,L,pi,T)!=rec_running(si,L,pi,T)
            print(f"                (P,pi) pairs separating a full cycle from its inverse: {w}/{t_} =
{100*w/t_:.1f}%")
            f"      static binary family: 0 (proved)")
30. =====
SECTION 3.  R1 consistent set = the double coset KsK (brute force, S_6)
P=[(0, 1, 2), (3, 4, 5)] sigma==+1: |consistent|= 324 |KsK|= 324 closed form= 324 match=True
P=[(0, 3), (1, 4), (2, 5)] sigma==+1: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1), (2, 3), (4, 5)] sigma==+2: |consistent|= 8 |KsK|= 8 closed form= 8 match=True
P=[(0, 1, 2), (3, 4, 5)] sigma==+3: |consistent|= 36 |KsK|= 36 closed form= 36 match=True
P=[(0, 2, 4), (1, 3, 5)] sigma==+1: |consistent|= 36 |KsK|= 36 closed form= 36 match=True

=====
SECTION 4.  the ladder G <= R3 <= R2 <= R1 (class reps x all set partitions)
n  configs  |R2|<|R1|  |R3|<|R2|  R3==G  R1==G  4  75  40  3
39 (52.0%)  5  5  364  262  18  201 (55.2%)  7  6  2233
1838  132  1205 (54.0%)  11  |R3|/|G| multiplicities: {1: 1205, 2: 553, 3: 41,
4: 174, 5: 2, 6: 80, 8: 44, 9: 1, 12: 72, 16: 5, 18: 3, 24: 18, 36: 10, 48: 18, 120: 6, 720: 1} (no
non-integer ratios: True)

=====
SECTION 5.  R1 floor = log2|K|, attained exactly on aligned partitions (Z/12)
sigma==+1 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma==+1 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma==+1 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

sigma=+1 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+2 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+2 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+2 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+2 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True
sigma=+3 labels=2: floor= 18.984 min= 18.984 median= 26.797 worst= 27.628 aligned-attains=True
sigma=+3 labels=3: floor= 13.755 min= 13.755 median= 23.510 worst= 24.510 aligned-attains=True
sigma=+3 labels=4: floor= 10.340 min= 10.340 median= 18.680 worst= 20.680 aligned-attains=True
sigma=+3 labels=6: floor= 6.000 min= 6.000 median= 12.000 worst= 12.000 aligned-attains=True

=====
SECTION 6. passive observation: consistent set vs the group-based figure (Z/12)
  <+1> orbits [12]: group-based 39916800 (25.250 b) consistent 39916800 (25.250 b) ratio 1.0x
  <+2> orbits [6, 6]: group-based 86400 (16.399 b) consistent 14400 (13.814 b) ratio 6.0x
  <+3> orbits [4, 4, 4]: group-based 3456 (11.755 b) consistent 216 ( 7.755 b) ratio
16.0x transitive coincidence: 12!/12 = 11! = True

=====
SECTION 7. joint reader census S_4..S_7, and the extremal family
  n configs gain max excess aligned among gainers
  4 75 2 +0.5850 0
  5 364 21 +1.0000 0
  6 2233 148 +1.5850 0
  7 13155 929 +2.3219 0
  extremal family lambda=(4,1^(n-4)), P={1,3}|rest, gain=(n-2)(n-3)/4 n=6: |KsK| 288/288
|M| 12/12 gain 3.000/3.000 excess +1.5850 b n=7: |KsK| 2400/2400 |M| 20/20 gain
5.000/5.000 excess +2.3219 b n=8: |KsK| 21600/21600 |M| 30/30 gain 7.500/7.500
excess +2.9069 b n=9: |KsK| 211680/211680 |M| 42/42 gain 10.500/10.500 excess +3.3923 b

=====
SECTION 8. families of readers, and the orientation floor n=5: complete binary family 15 readers
(= 2^(n-1)-1 = 15) -> 2 candidates; == {sigma, sigma^-1}: True n=6: complete binary family 31
readers (= 2^(n-1)-1 = 31) -> 2 candidates; == {sigma, sigma^-1}: True n=7: complete binary
family 63 readers (= 2^(n-1)-1 = 63) -> 2 candidates; == {sigma, sigma^-1}: True n=5: 3-block
readers breaking the tie: 15/25; minimum proper partitions for exact recovery = 2 witness:
[[{0,}, {1,}, {2, 3, 4}], [{0, 2}, {3,}, {1, 4}]]
n=6: 3-block readers breaking the tie: 51/90; minimum proper partitions for exact recovery = 2
witness: [[{0,}, {1,}, {2, 3, 4, 5}], [{0, 2}, {3,}, {4,}, {1, 5}]]

=====
SECTION 3 / 4A. the record IS the contingency table; product formula |KsK| = (prod|B|!)^2 / prod
N_ij! in all 6600 pairs at n=4,5 : True worked example reader A: N=((2, 1), (1, 2)) |K|=36
|KsK|=324 worked example reader B: N=((0, 3), (3, 0)) |K|=36 |KsK|=36

=====
SECTION 4. inversion criterion: sigma^-1 in KsK <=> N(P,sigma) symmetric
31.

```